

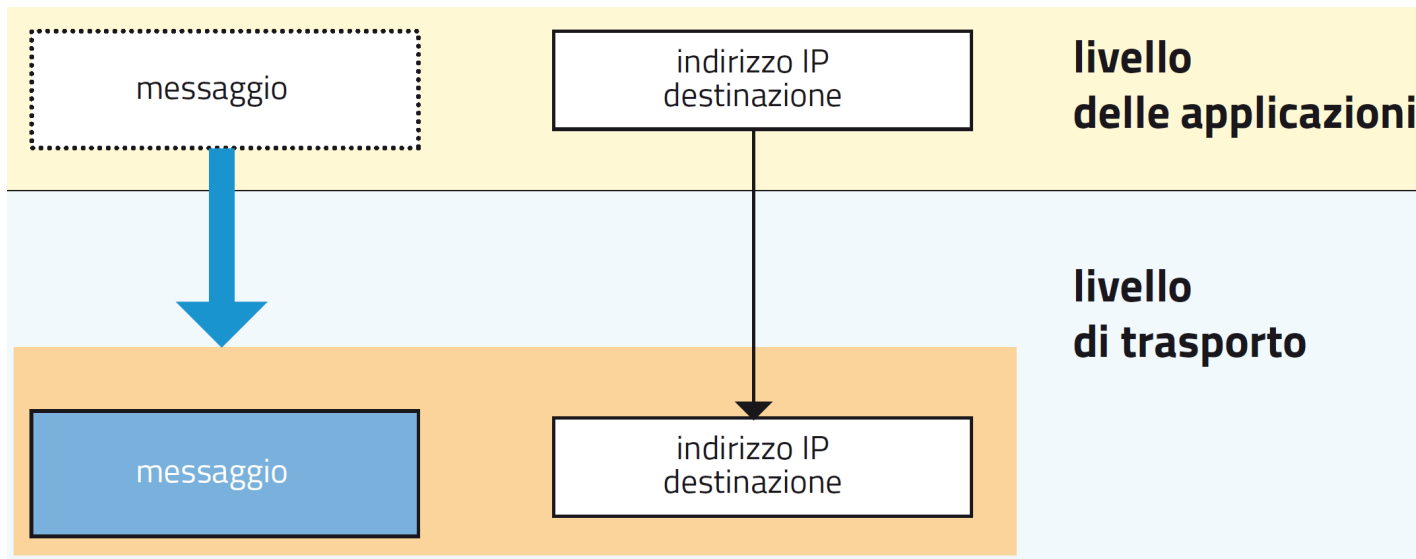
capitolo 4 – Il livello di trasporto e il livello di applicazione

1. I protocolli del livello di trasporto
2. Il livello di applicazione
3. Il protocollo HTTP
4. Trasferire file: il protocollo FTP
5. La posta elettronica
6. Il DNS

4.1 I protocolli del livello di trasporto

Indice

Un'**applicazione di rete**, per comunicare con un'applicazione **di pari livello** in un **host remoto**, affida i propri messaggi al **livello di trasporto**.



Il **livello di trasporto** garantisce una comunicazione di tipo **end-to-end**: fornisce i propri servizi esclusivamente nei due **nodi terminali** (sorgente e destinazione); in quelli intermedi è interessato soltanto il livello IP della suite.

4.1 I protocolli del livello di trasporto

Indice

Un'**applicazione di rete**, per comunicare con un'applicazione **di pari livello** in un **host remoto**, affida i propri messaggi al **livello di trasporto**.

Il **livello di applicazione** può richiedere al **livello di trasporto** due tipi di servizi, che sono tra loro alternativi:

- Il **TCP** (*Transmission Control Protocol*) è un protocollo che privilegia l'**affidabilità** a discapito della **velocità di trasmissione** dei messaggi: stabilisce comunicazioni **connection-oriented** (dette anche di tipo *stream*);
- L'**UDP** (*User Datagram Protocol*) è invece un protocollo che garantisce una maggiore velocità a spese dell'affidabilità della trasmissione, perché questa avviene in modalità **connectionless** (o di tipo *messaggio*).

4.1 I protocolli del livello di trasporto

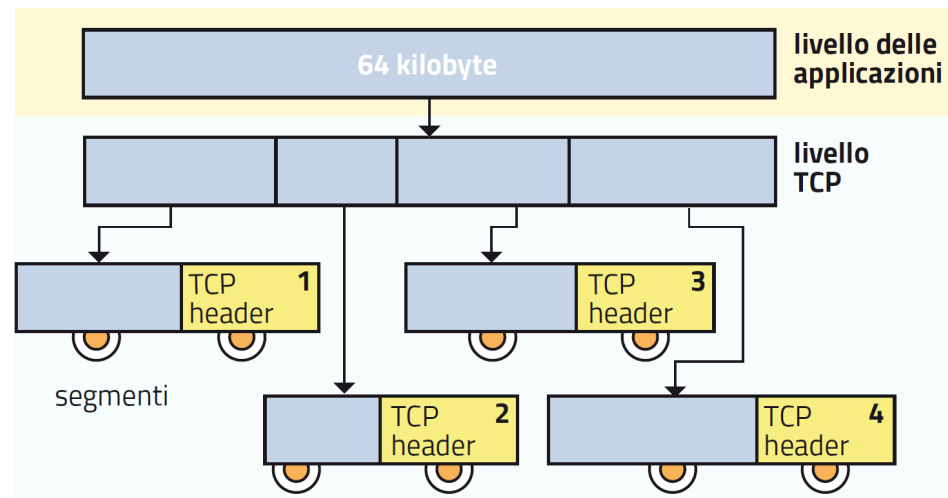
Indice

Il **protocollo TCP** (*Transmission Control Protocol*) è **affidabile** perché opera in modalità **connection-oriented**.

Prima di cominciare la trasmissione, il **protocollo TCP** stabilisce una **connessione logica** con l'host remoto, per mezzo di uno scambio iniziale di messaggi detto **handshake**, cioè «stretta di mano».

Il TCP riceve dal livello delle applicazioni 64 kbyte alla volta e li divide in blocchi più piccoli.

A ciascun blocco poi aggiunge un'intestazione (*TCP header*), formando così un pacchetto chiamato **segmento**.



4.1 I protocolli del livello di trasporto

Indice

Il **protocollo TCP** (*Transmission Control Protocol*) è **affidabile** perché opera in modalità **connection-oriented**.

I **segmenti sono numerati** nel campo **sequence number** del TCP header per permettere al **TCP ricevente** di **ordinarli nella giusta sequenza**, **ricostruendo il messaggio** nella sua interezza.

Il TCP ricevente specifica con il **messaggio di risposta ACK** anche il **numero del segmento che si aspetta di ricevere con il messaggio successivo**.

In questo modo conferma implicitamente di aver ricevuto correttamente i segmenti precedenti.

I segmenti di cui non si è ricevuto l'acknowledgment vanno invece **ritrasmessi**.

Il **protocollo TCP** controlla se i segmenti in arrivo siano danneggiati.

Il controllo è reso possibile dall'algoritmo detto **checksum**:

- il TCP header lato trasmittente contiene un campo con il complemento a 1 della somma dei complementi a 1 di tutte le parole di 16 bit presenti nell'intestazione stessa e nel campo **dati**;
- il TCP ricevente effettua lo stesso calcolo e confronta il risultato con il contenuto del campo **checksum** del segmento ricevuto; se i due non coincidono, non invia il relativo acknowledgment, nel qual caso il segmento sarà rispedito.

4.1 I protocolli del livello di trasporto

Indice

Il TCP regola la velocità di trasmissione secondo la capacità dell'host ricevente e tenendo conto di eventuali congestioni dei nodi intermedi.

Dopo ogni acknowledgment, il TCP trasmittente regola la propria **finestra di trasmissione** o **window**, inviando il numero di segmenti che il TCP remoto è in grado di ricevere grazie alla sua capacità di memorizzazione

La finestra però si può restringere nel corso della trasmissione, nel caso si verifichi una **congestione nei nodi intermedi**, cioè nei router.

Infatti il TCP trasmittente, se non riceve un acknowledgment entro lo scadere del timeout, **restringe la finestra** trasmettendo un solo segmento. Alla trasmissione successiva la allarga, trasmettendo due segmenti; poi raddoppia ogni volta.

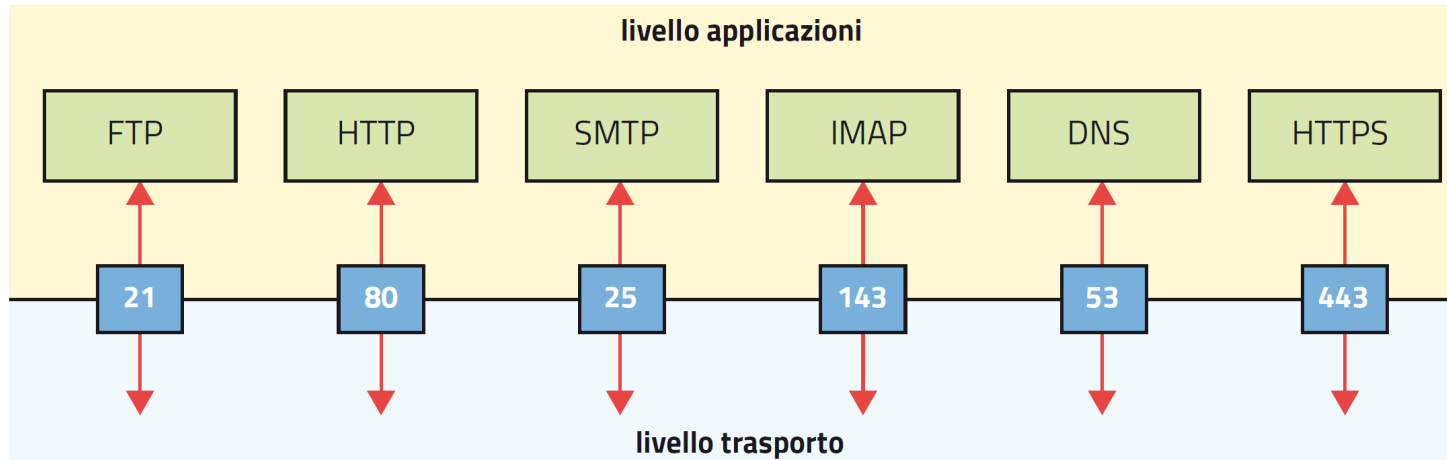
In caso di nuovo timeout, il TCP trasmittente torna a restringere la finestra trasmettendo un solo segmento, poi due, poi quattro e così via.

Questa procedura, che provoca una successione di velocizzazioni e rallentamenti nella trasmissione, è chiamata **windowing**.

4.1 I protocolli del livello di trasporto

Indice

Il **TCP** identifica il **protocollo del livello superiore** per mezzo di un **numero di porta** presente in un campo dell'**header**.



La **coppia [indirizzo IP / numero di porta]** è chiamata **socket**.

Ogni comunicazione dello strato TCP/IP è sempre tra due socket, uno sorgente e l'altro di destinazione.

4.1 I protocolli del livello di trasporto

Certe porte nei server sono sempre associate alle applicazioni più comuni.

Le comunicazioni tra due host in rete avvengono sempre in seguito a una **richiesta di servizio** da parte di un **client** a un **server**, che si suppone sempre in ascolto. È necessario perciò che il client conosca in anticipo il **numero di porta** dell'applicazione residente nel server.

La **IANA** (*Internet Assigned Number Authority*) ha definito le **well-known ports**, «porte ben conosciute», che vanno dalla numero 0 alla numero 1023.

numero di porta	applicazione
20	FTP data transfer
21	FTP control
23	Telnet
25	SMTP (Simple Mail Transfer Protocol)
53	DNS (Domain Name System)
80	HTTP (HyperText Transfer Protocol)
110	POP3 (Post Office Protocol 3)
143	IMAP (Internet Message Access Protocol)
443	HTTPS (Secure HTTP)

4.1 I protocolli del livello di trasporto

Indice

I **client** usano **numeri di porta casuali** tra quelli liberi: non è necessario che il server li conosca in anticipo.

Le **porte** con numeri compresi tra **1024** e **49151** sono sostanzialmente libere, ma i produttori dell'hardware possono riservarle per usi specifici; invece le porte restanti, **dalla 49 152 alla 65 536**, si possono usare senza restrizioni.

I **client** normalmente utilizzano i propri numeri di porta **dal 1024 in poi** in piena libertà, perché non occorre che il server li conosca in anticipo.

Infatti è **sempre il client a iniziare la comunicazione**, con una richiesta in cui **indica il numero di porta sorgente** che il server potrà utilizzare nei successivi messaggi di risposta.

4.1 I protocolli del livello di trasporto

Indice

Il **protocollo UDP** (*User Datagram Protocol*) non effettua la correzione degli errori: opera cioè in **modalità connectionless** ed è quindi **più veloce**.

Alcune applicazioni di rete usano l'**UDP** perché necessitano di comunicazioni rapide e possono tollerare la conseguente perdita di affidabilità. **Esempi:**

- il **DNS** (*Domain Name System*), protocollo che all'inizio della comunicazione tra due applicazioni di rete ha il compito di tradurre l'indirizzo simbolico della risorsa richiesta al server nel corrispondente indirizzo IP di destinazione.
- l'**RTP** (*Real Time Protocol*), usato nel corso delle chiamate di tipo **VoIP** (*Voice over Internet Protocol*), molto comuni con gli smartphone.

4.1 I protocolli del livello di trasporto

Il **TCP header** contiene informazioni indispensabili per la buona riuscita della comunicazione.

L'**header TCP** contiene, tra altri, i seguenti campi:

campo	significato
Source port	è il numero di porta dell'applicazione sorgente, che servirà al destinatario per la risposta
Destination port	è il numero di porta dell'applicazione di destinazione, che servirà al livello di trasporto del destinatario per sapere a quale applicazione consegnare il messaggio in arrivo
Sequence number	è il numero del segmento, che servirà al destinatario per riordinare i segmenti
ACK number	è il numero dell'ultimo segmento ricevuto correttamente; serve per far sapere al destinatario che nel prossimo messaggio dovrà trasmettere i segmenti a partire da quello successivo a questo
Checksum	è la somma di controllo, con cui il destinatario potrà verificare la correttezza del messaggio

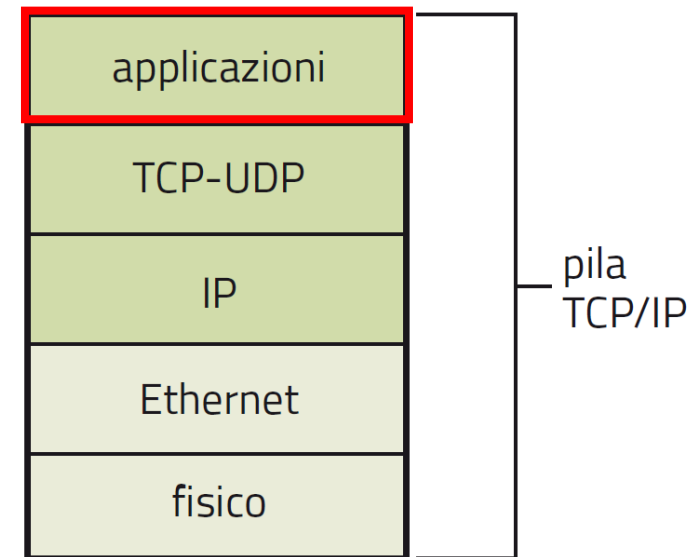
4.2 Il livello di applicazione

Indice

Il **livello applicazione** ospita i protocolli che gestiscono **applicazioni di rete**.

Quando un utente di Internet richiede servizi offerti dalla Rete – come **la navigazione nel web**, l'invio o la ricezione di messaggi di **posta elettronica** oppure il **download di file** – il suo computer attiva un'apposita **applicazione di rete**.

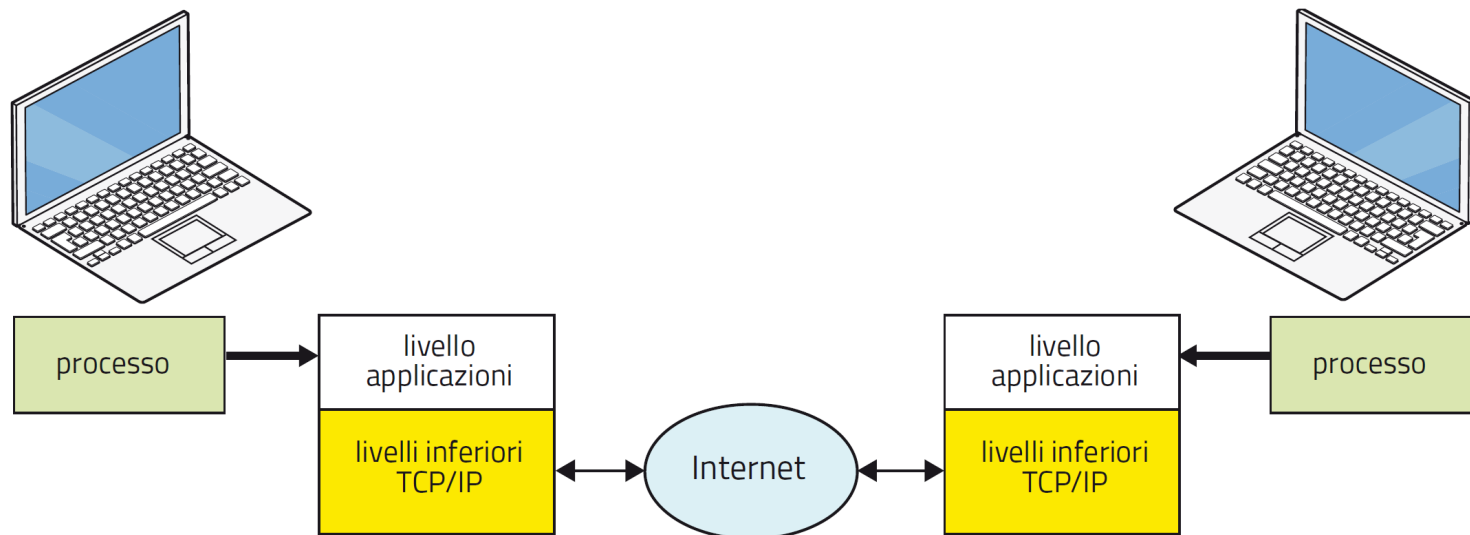
I protocolli che gestiscono le applicazioni di rete si trovano al livello più alto della cosiddetta *pila* o *stack* della suite TCP/IP, cioè al **livello delle applicazioni**, chiamato anche semplicemente **livello applicazione**.



4.2 Il livello di applicazione

Le **applicazioni di rete** sono applicazioni **distribuite**, perché la loro comunicazione coinvolge **processi contemporanei** su più host della rete.

I diversi **processi**, per comunicare, stabiliscono una connessione per mezzo degli **indirizzi IP** degli host, usando i servizi di comunicazione offerti dal protocollo del livello di applicazione della pila TCP/IP.



4.2 Il livello di applicazione

Quando un utente **richiede un servizio** alla Rete, si attiva il corrispondente **protocollo del livello applicazione**.

Il **livello di applicazione** fornisce diversi servizi a chi si collega alla Rete:

- **HTTP** (*HyperText Transfer Protocol*) per il trasferimento di pagine web;
- **SMTP** (*Simple Mail Transfer Protocol*) per l'invio di messaggi di posta elettronica;
- **FTP** (*File Transfer Protocol*) per il trasferimento di file;
- **Telnet** (*Teletype network*) per operare su una macchina remota come se fosse in locale.

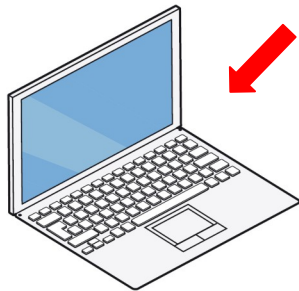
Altre applicazioni di rete non sono direttamente richiamate dall'utente, ma fungono da supporto alla comunicazione:

- **DNS** (*Domain Name System*) per la traduzione di un nome simbolico (come un indirizzo web o un nome di computer) in un indirizzo IP;
- **DHCP** (*Dynamic Host Configuration Protocol*) per assegnare indirizzi IP in modo dinamico.

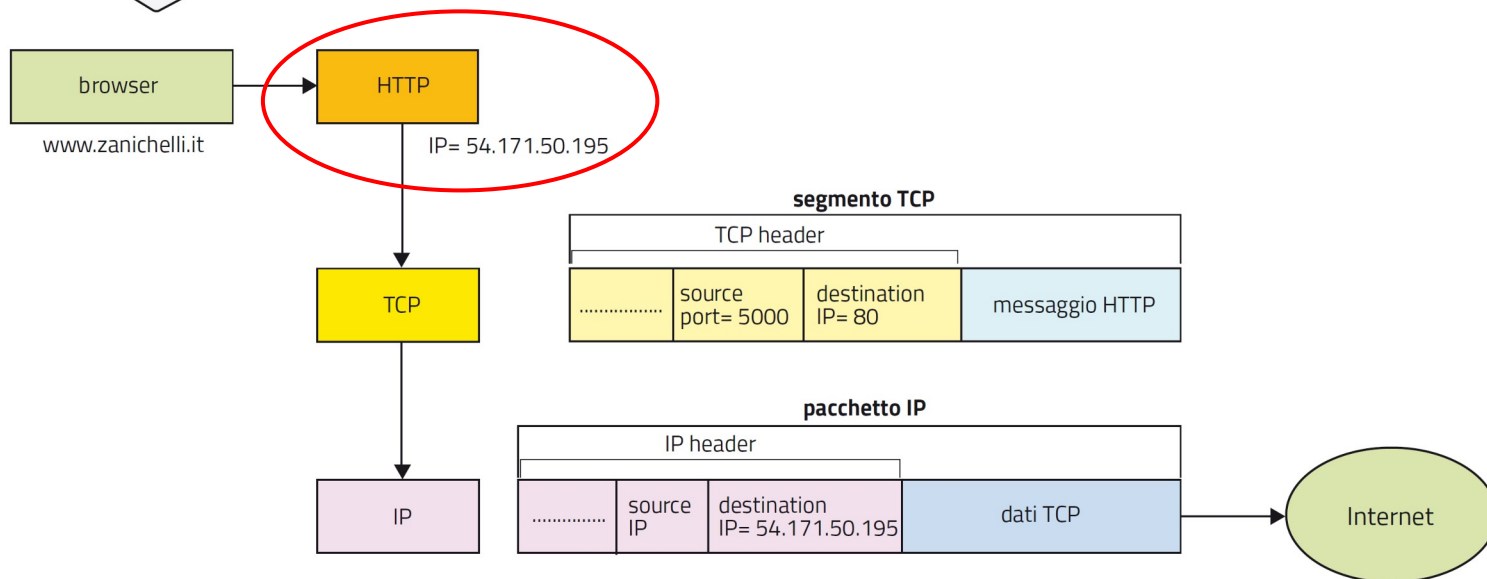
4.2 Il livello di applicazione

Un esempio di comunicazione tra applicazioni di rete.

Un utente vuole collegarsi al sito web della Zanichelli e digita **www.zanichelli.it** nella barra degli indirizzi del browser.



Il browser attiva l'applicazione HTTP; questa fornirà alla pila TCP/IP (del sistema operativo dell'utente) l'indirizzo IP del server che vuole raggiungere.



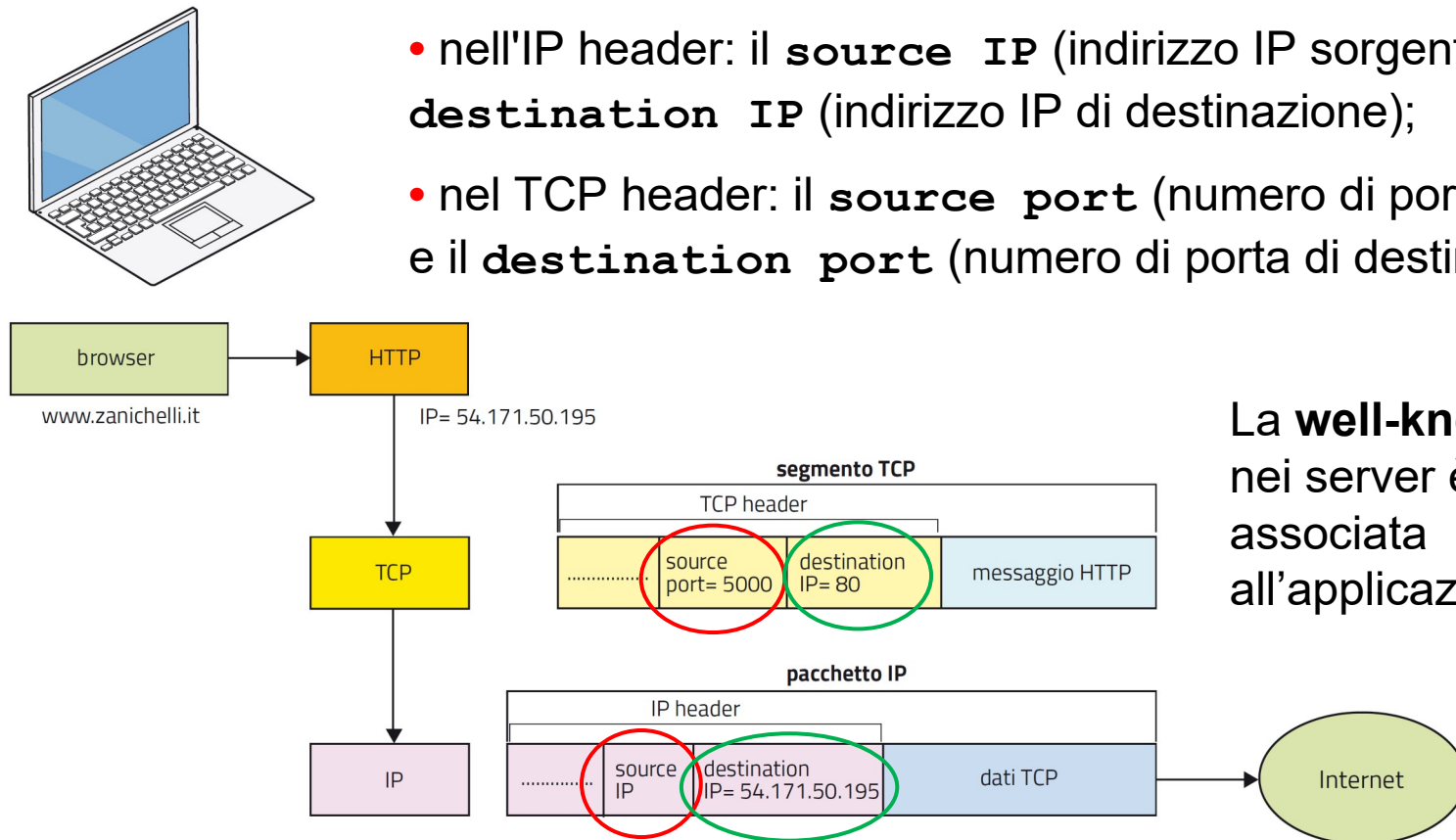
4.2 Il livello di applicazione

Indice

Un esempio di comunicazione tra applicazioni di rete.

I protocolli TCP e IP prepareranno un pacchetto che contiene:

- nell'IP header: il **source IP** (indirizzo IP sorgente) e il **destination IP** (indirizzo IP di destinazione);
- nel TCP header: il **source port** (numero di porta sorgente) e il **destination port** (numero di porta di destinazione);



La **well-known port 80** nei server è sempre associata all'applicazione HTTP.

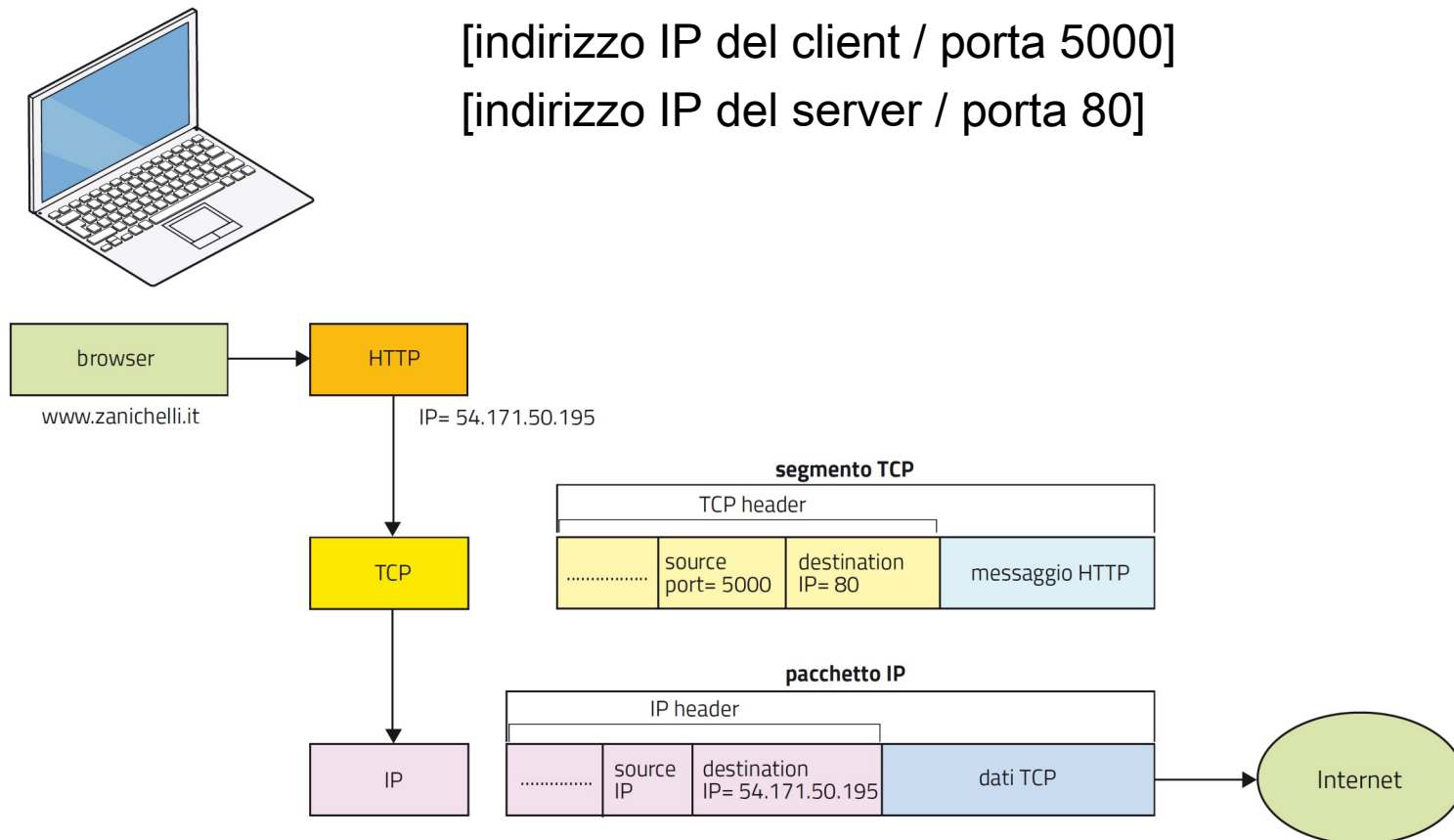
4.2 Il livello di applicazione

Un **esempio di comunicazione tra applicazioni di rete.**

Avremo così due **socket**:

[indirizzo IP del client / porta 5000]

[indirizzo IP del server / porta 80]



4.2 Il livello di applicazione

Un esempio di comunicazione tra applicazioni di rete.

Il pacchetto così confezionato viaggia attraverso Internet e raggiunge il server:

- lo stack TCP/IP del server analizza il **destination IP**, che riconoscerà come proprio;
- quindi analizza la **destination port 80**, che è nello stato di **ascolto** (*listening*), e il segmento è affidato a un'applicazione come **Apache** (o un altro web server, per esempio **IIS**), che nel server è l'omologa del browser;
- Apache vede che è stata richiesta la pagina **www.zanichelli.it** e per rispondere al client chiede allo stack TCP/IP di confezionare un pacchetto di risposta, nel quale **i due indirizzi IP sorgente e di destinazione saranno invertiti, così come i due numeri di porta.**

4.2 Il livello di applicazione

Indice

Un **esempio di comunicazione tra applicazioni di rete.**

Se adesso, mentre è in atto la comunicazione tra browser e Apache per mezzo del protocollo HTTP per la navigazione web, il client richiedesse il servizio FTP per il trasferimento di un file, il messaggio in partenza dal client avrebbe come **destination IP** sempre l'indirizzo del server, **ma come porta la numero 21.**

Il server, ricevendo il pacchetto, si accorgerebbe che la comunicazione è destinata al proprio indirizzo IP, ma siccome la porta è la numero 21 affiderebbe il messaggio non più ad Apache o a IIS, ma a un'**applicazione FTP server.**

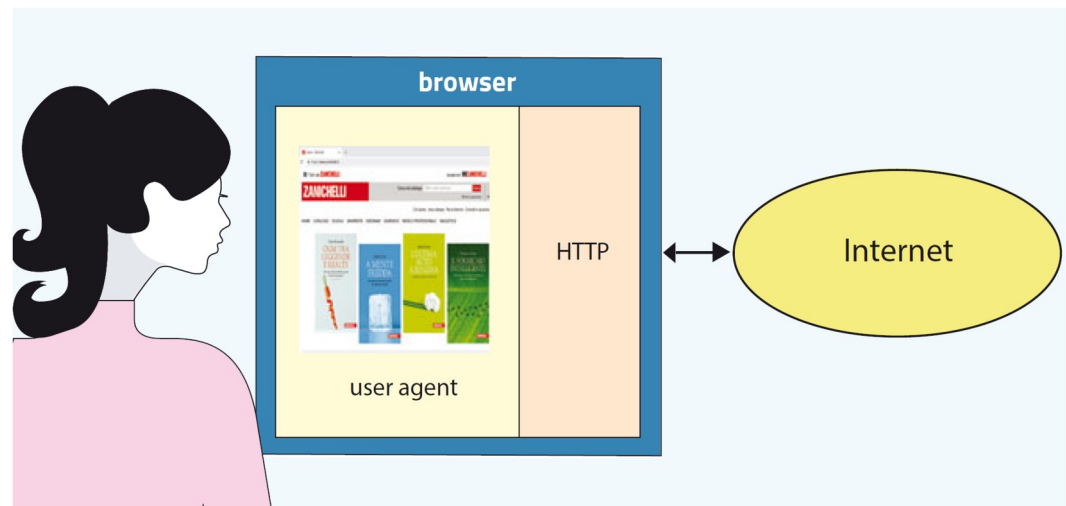
4.2 Il livello di applicazione

Un'**applicazione di rete** deve fornire a un **utente** la possibilità di **usufruire dei servizi di Internet**.

Ogni applicazione ha due funzionalità:

- **user agent**: comunica con l'utente fornendogli un'interfaccia grafica di facile uso;
- **motore**: implementa i protocolli di comunicazione per interagire con i server presenti in rete, i quali dovranno fornire i servizi richiesti.

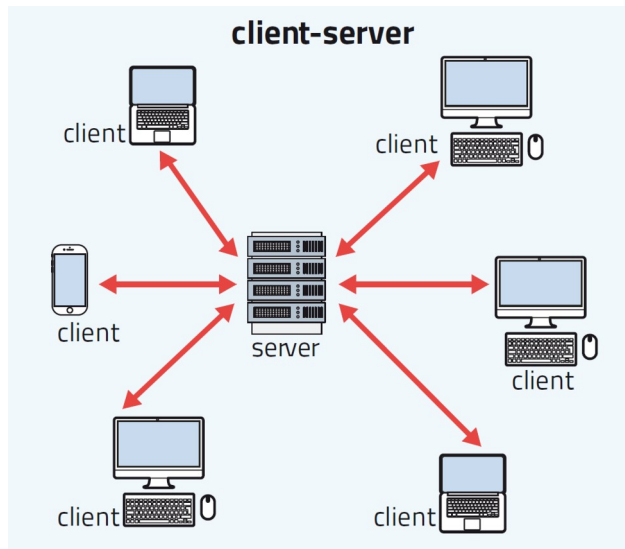
Esempio:
il **browser lato utente** permette di indicare l'indirizzo web, mentre **lato rete** implementa il protocollo HTTP per lo scambio di messaggi con i server.



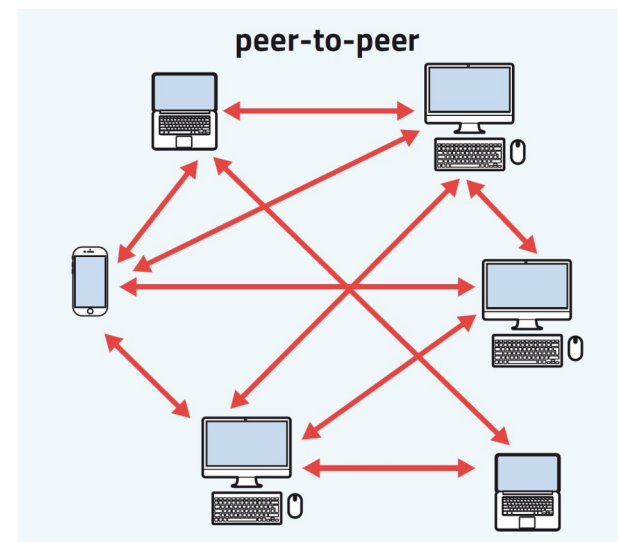
4.2 Il livello di applicazione

Per **progettare una nuova applicazione di rete** bisogna tenere conto del tipo di **architettura di rete** cui fare riferimento.

Architettura **client-server**: un servizio di rete è fornito da un host chiamato **server**, sempre in ascolto, dopo richiesta da parte di uno o più host chiamati **client**.



Architettura **peer-to-peer**: ciascun host può agire **sia da client sia da server**; a volte un host richiederà un servizio a un altro host, altre volte potrà fornirgli il medesimo servizio.



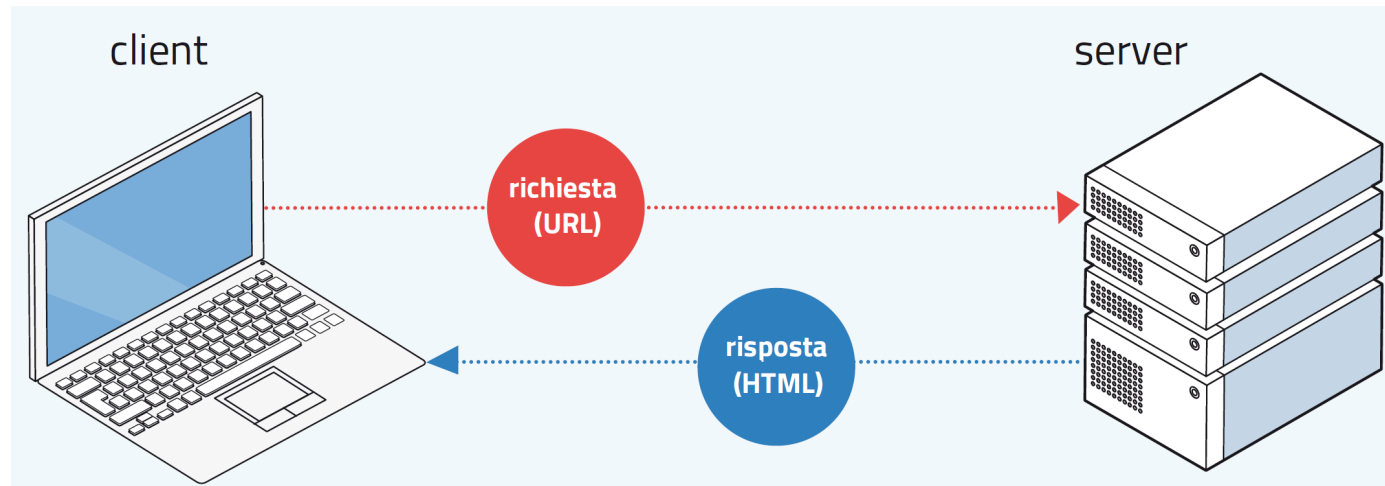
4.3 Il protocollo HTTP

Indice

Quando un utente digita l'indirizzo di una **pagina web**, il **browser** usa il **protocollo HTTP** per **comunicare con il server** che la ospita.

L'**HTTP** (*HyperText Transfer Protocol*) è un protocollo di comunicazione che risiede nel **livello applicazioni** sia del client sia del server.

Il **browser** analizza il comando dell'utente e **richiede la pagina web al server** interessato, specificandone l'**URL** (*Uniform Resource Locator*), indirizzo che individua la pagina web.

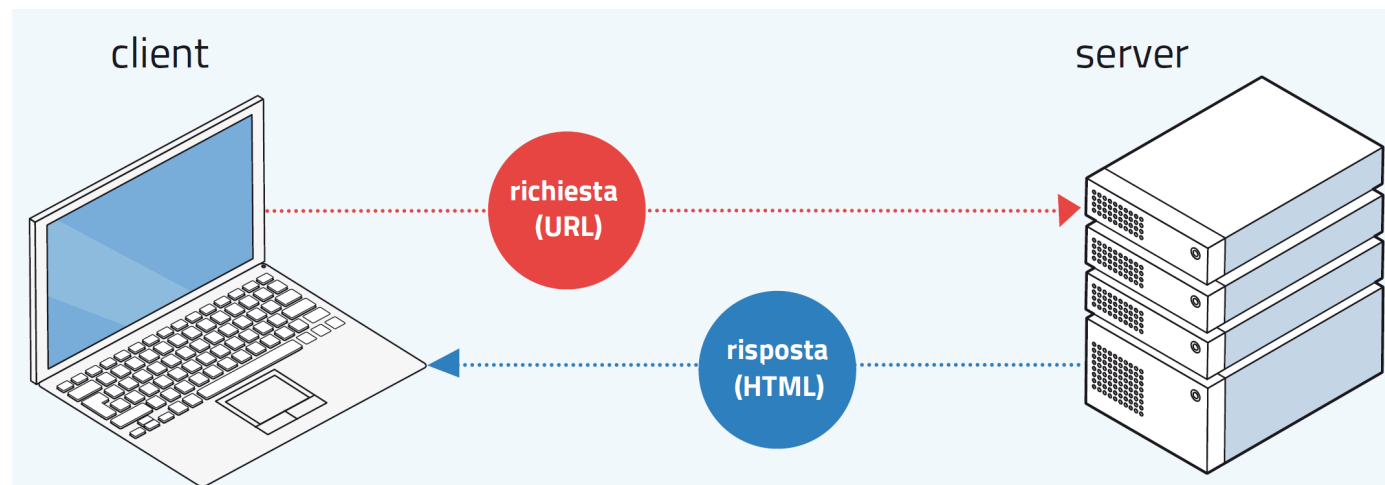


4.3 Il protocollo HTTP

Quando un utente digita l'indirizzo di una **pagina web**, il **browser** usa il **protocollo HTTP** per **comunicare con il server** che la ospita.

Il **server risponde** inviando al client:

- un file in codice **HTML** (*HyperText Markup Language*) puramente descrittivo che il browser interpreta per ricostruire la pagina web sul dispositivo dell'utente;
- un file in codice **CSS** (*Cascading Style Sheets*) con indicazioni sugli stili da usare nella pagina;
- tutti gli ulteriori dati che servono per costruire la pagina, come immagini, video, script e applet Java.



4.3 Il protocollo HTTP

Indice

L'**Uniform Resource Locator (URL)** è l'indirizzo che **individua univocamente** ogni **oggetto** presente nel web: pagina, immagine, documento pdf e così via.

L'**URL** è l'indirizzo che compare nella **barra degli indirizzi del browser** quando siamo collegati a una **pagina web**, ed è composto da diverse sezioni.

Esempio:

https://www.zanichelli.it/scuola/in-primo-piano

- **https** è il **protocollo** usato per la comunicazione con il server (la lettera **s**, che sta per *secure*, compare perché in questo caso la comunicazione è criptata e al protocollo HTTP è aggiunto un livello di codifica dei dati);
- **www.zanichelli.it** è il nome simbolico che identifica il **web server** su cui risiedono i dati;
- **/scuola/in-primo-piano** individua il **percorso** da seguire all'interno del server per raggiungere la risorsa desiderata.

4.3 Il protocollo HTTP

Indice

Esempio: l'apertura della pagina all'URL
<https://www.zanichelli.it/scuola/in-primo-piano>.

Per costruire questa pagina sullo schermo di un pc o di uno smartphone, il browser avrà bisogno di:

- un **file HTML**, che descrive la disposizione degli oggetti sullo schermo;
- un **file CSS**, che determina lo stile della pagina (grafica delle voci di menu, titoli, formattazioni dei testi e così via);
- **altri file** con le immagini da inserire nelle posizioni specificate dal file HTML.

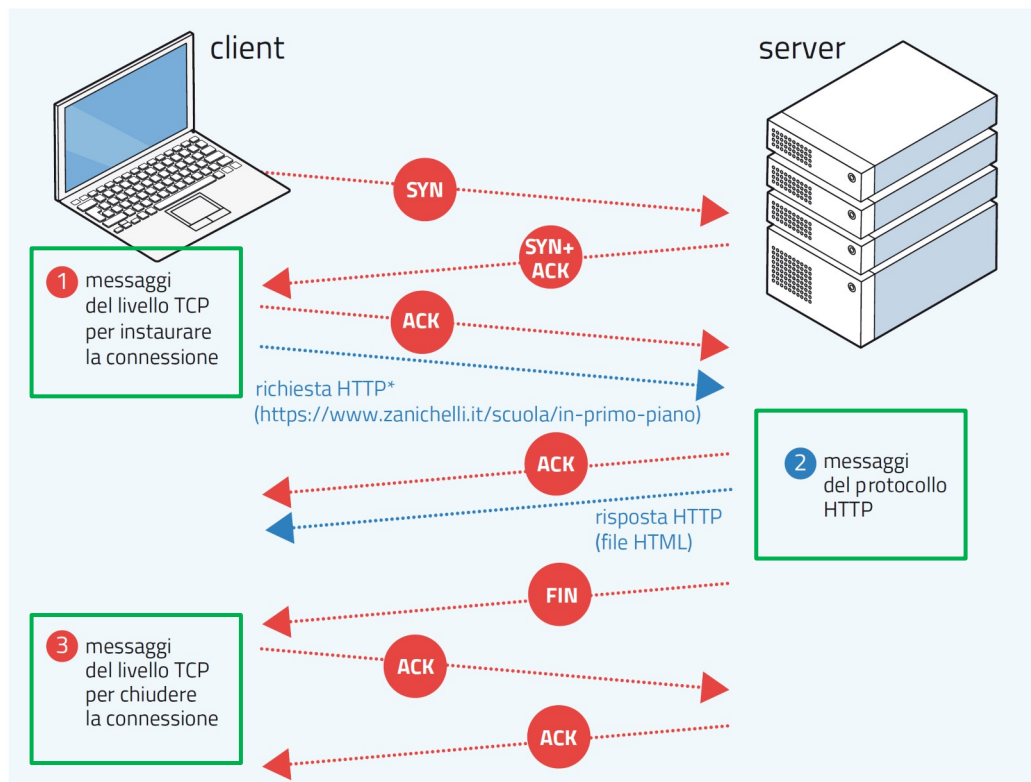


4.3 Il protocollo HTTP

Esempio: l'apertura della pagina all'URL
<https://www.zanichelli.it/scuola/in-primo-piano>.

Per **ricevere le risorse** elencate, la comunicazione tra il server e il client – che per trasferire i messaggi userà lo **strato TCP** – sarà di questo tipo:

1. il client richiede la connessione con la porta 80 del server, che accetta;
2. il client richiede la pagina web e il server risponde inviando il file HTML;
3. il server notifica al TCP la chiusura della connessione, che avverrà dopo che il messaggio sarà stato riscontrato dal client.



4.3 Il protocollo HTTP

Indice

Esempio: l'apertura della pagina all'URL

<https://www.zanichelli.it/scuola/in-primo-piano>.

Il client **dopo la chiusura della connessione** analizza il file HTML e vi trova i riferimenti al file CSS e alle immagini presenti nella pagina.

Attiva quindi una nuova connessione, composta dagli stessi passi da **1** a **3**, per ricevere il file CSS. La procedura è poi ripetuta **per ciascuna delle immagini**.

Le **diverse connessioni** possono però essere attivate **in parallelo**, il che velocizza i tempi di acquisizione dell'intera pagina web.

Soltanto quando il browser avrà ricevuto gli elementi indispensabili per la visualizzazione della pagina, essa apparirà sullo schermo dell'utente.

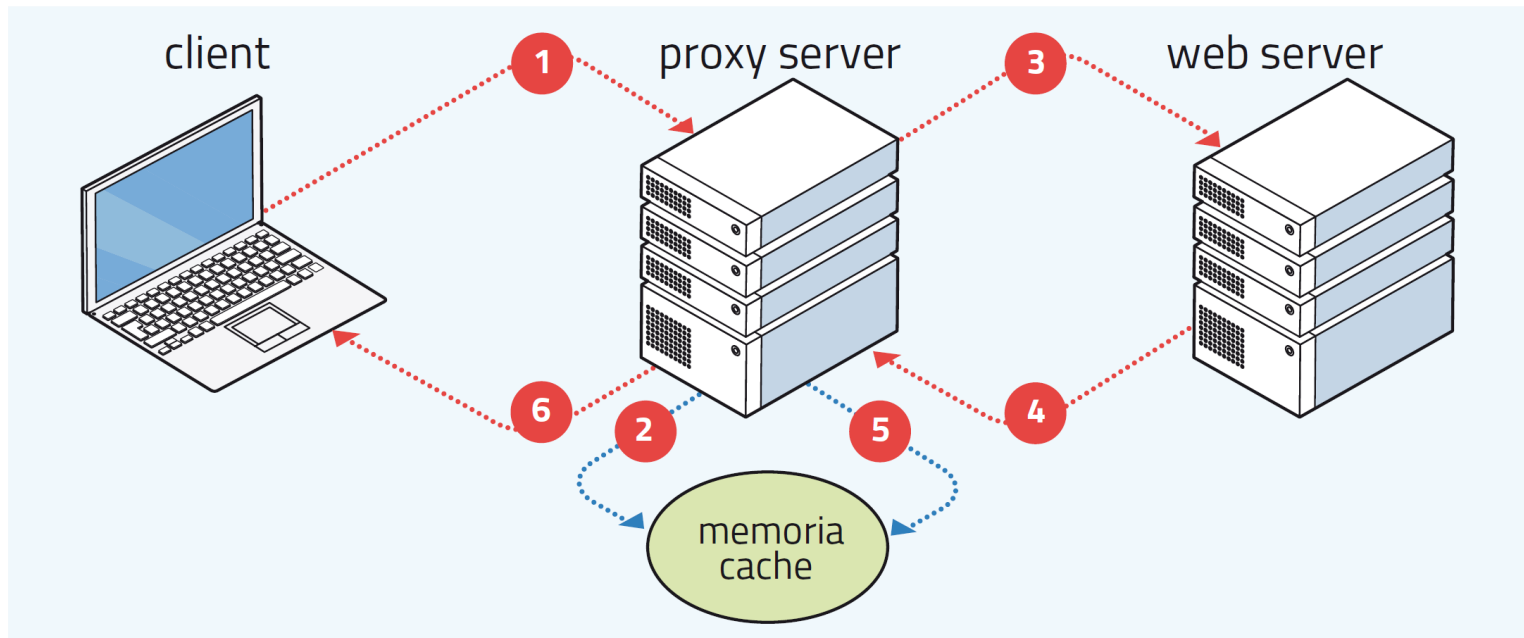
La più recente **versione 1.1** del protocollo HTTP prevede invece **connessioni persistenti**: il server non chiude la connessione dopo ogni risposta, ma la tiene attiva fino allo scadere di un periodo di timeout, così da consentire al client di effettuare tutte le richieste senza dover riattivare ogni volta la connessione.

4.3 Il protocollo HTTP

Indice

Per **ridurre il traffico su Internet** e i **tempi di trasmissione** delle risorse richieste dai vari client, la Rete è provvista di **proxy server**.

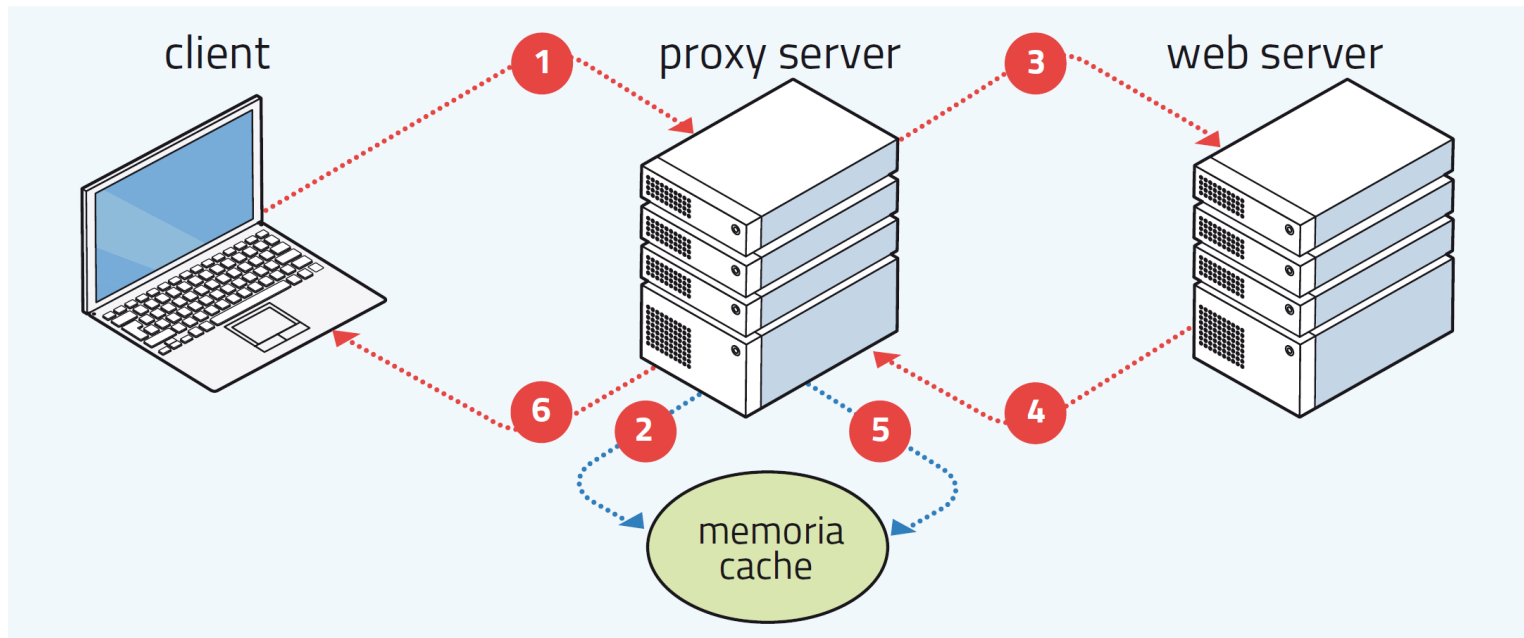
Questi **proxy server** («server delegati»), chiamati anche **cache web** («web in un nascondiglio»), si trovano in una posizione geografica non distante dal client e hanno la funzione di intercettare le richieste di quest'ultimo.



4.3 Il protocollo HTTP

Per **ridurre il traffico su Internet** e i **tempi di trasmissione** delle risorse richieste dai vari client, la Rete è provvista di **proxy server**.

1. quando un **client** richiede una **pagina web**, prima di tutto stabilisce una **connessione TCP** con il proprio **proxy server** predefinito;
2. se la pagina è presente nella sua **memoria cache**, il proxy la invia al client senza contattare il web server;

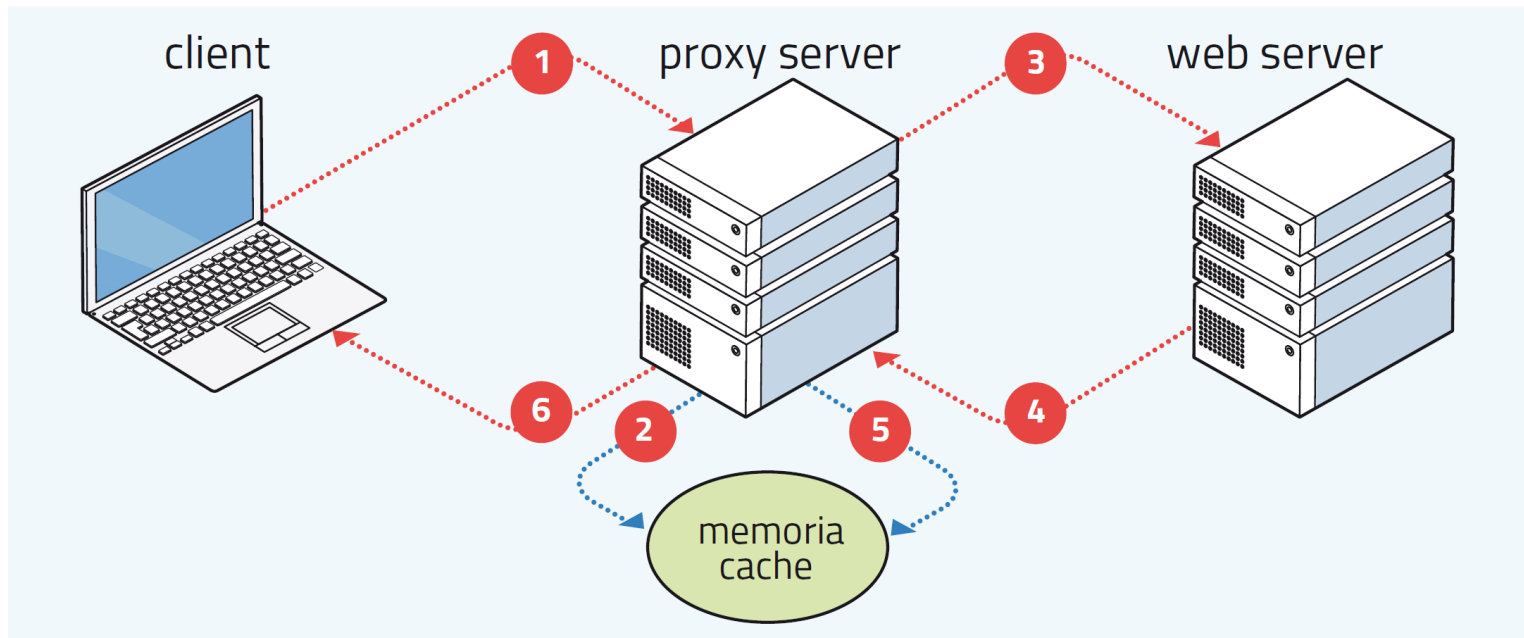


4.3 Il protocollo HTTP

Indice

Per **ridurre il traffico su Internet** e i **tempi di trasmissione** delle risorse richieste dai vari client, la Rete è provvista di **proxy server**.

3. altrimenti il proxy stabilisce una connessione TCP verso il **server di origine della pagina** e gli richiede la **risorsa**;
4. il web server in tal caso invia la pagina al proxy server;

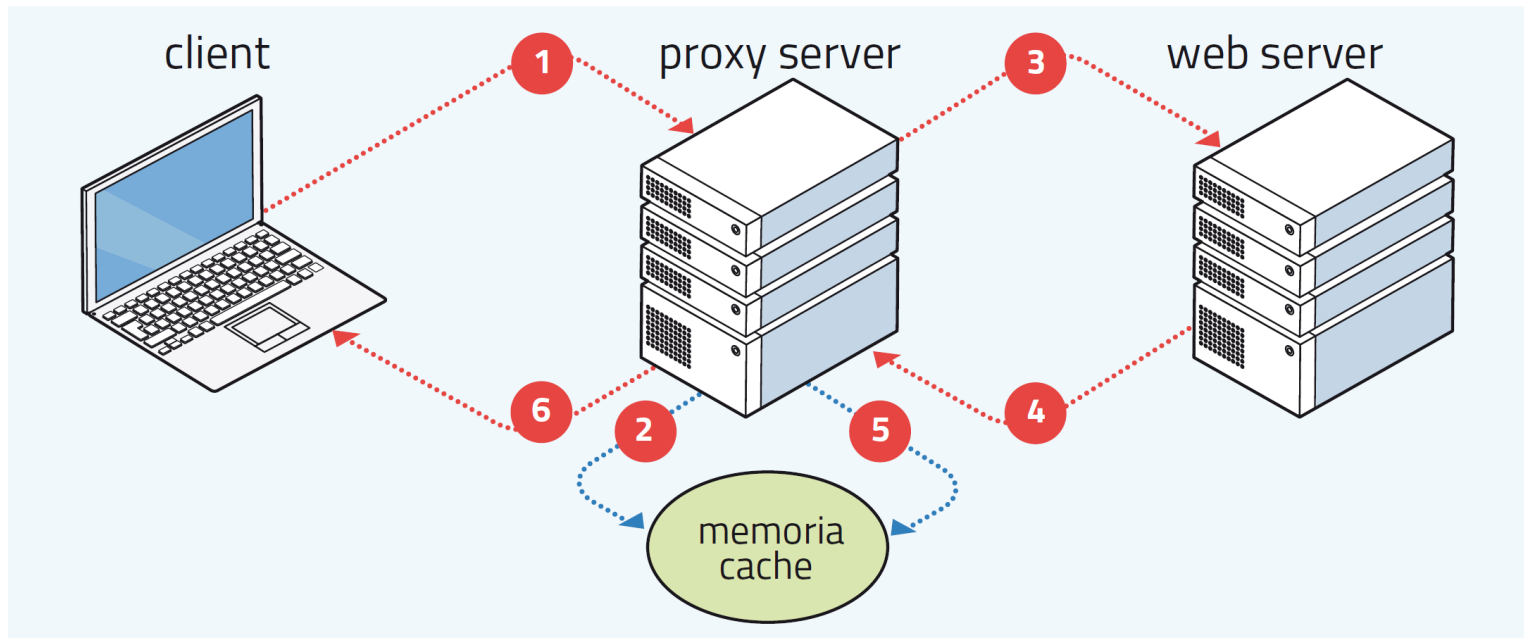


4.3 Il protocollo HTTP

Indice

Per **ridurre il traffico su Internet** e i **tempi di trasmissione** delle risorse richieste dai vari client, la Rete è provvista di **proxy server**.

5. il proxy memorizza la pagina nella propria memoria cache, così da averla a disposizione nel caso sia richiesta di nuovo in futuro;
6. infine il proxy inoltra la pagina web al client che l'aveva richiesta.

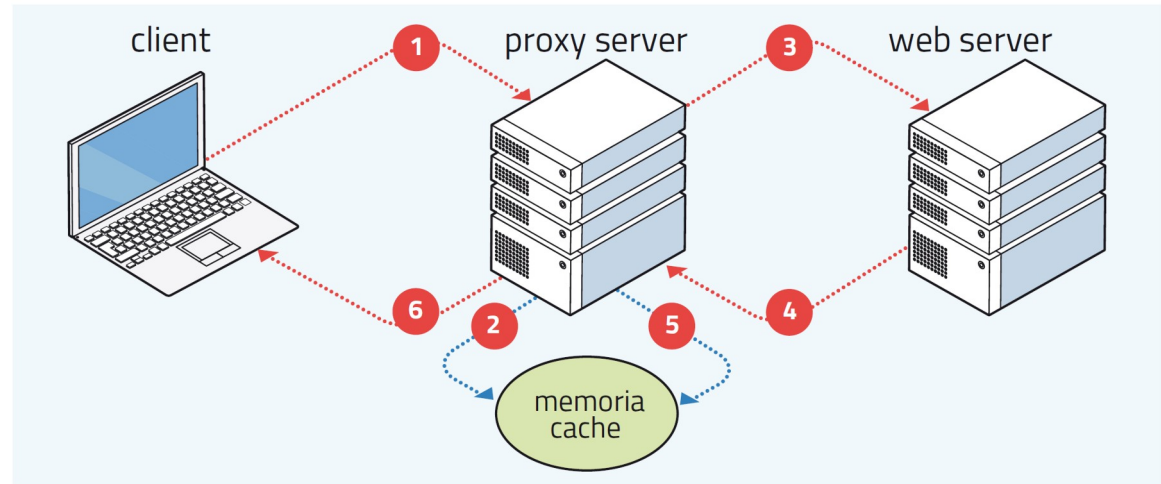


4.3 Il protocollo HTTP

Indice

Per **ridurre il traffico su Internet** e i **tempi di trasmissione** delle risorse richieste dai vari client, la Rete è provvista di **proxy server**.

Il **proxy** può agire sia da **client** (fase **3**) sia da **server** (fase **2** oppure fase **6**).



Quando la pagina web richiesta è presente nella memoria cache, perché un client geograficamente vicino al proxy l'ha visitata di recente, il proxy server **salta i passi 3-5** della procedura e risponde direttamente al client, **senza inoltrare la richiesta al web server**.

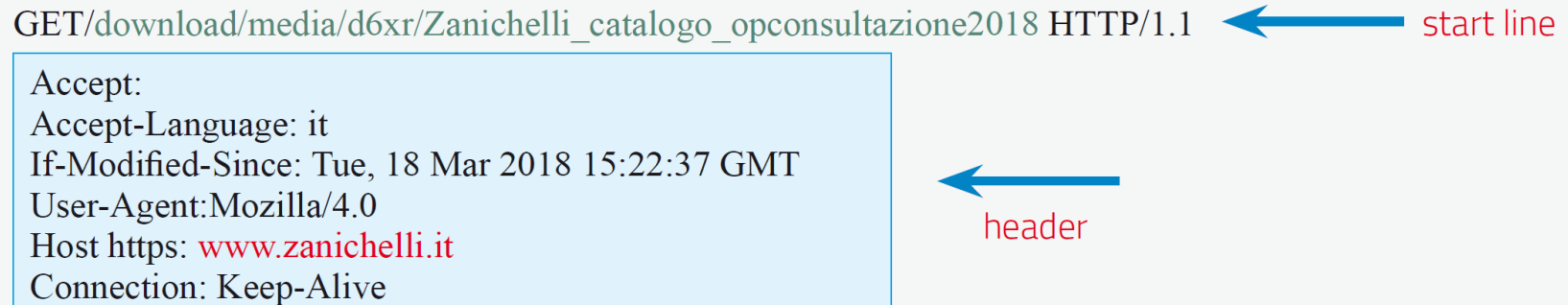
L'uso dei proxy contribuisce a ridurre il traffico dei dati in Rete perché fa diminuire le interrogazioni ai server geograficamente più lontani dai client.

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).

```
GET/download/media/d6xr/Zanichelli_catalogo_opconsultazione2018 HTTP/1.1
```



start line

```
Accept:  
Accept-Language: it  
If-Modified-Since: Tue, 18 Mar 2018 15:22:37 GMT  
User-Agent: Mozilla/4.0  
Host https: www.zanichelli.it  
Connection: Keep-Alive
```

header

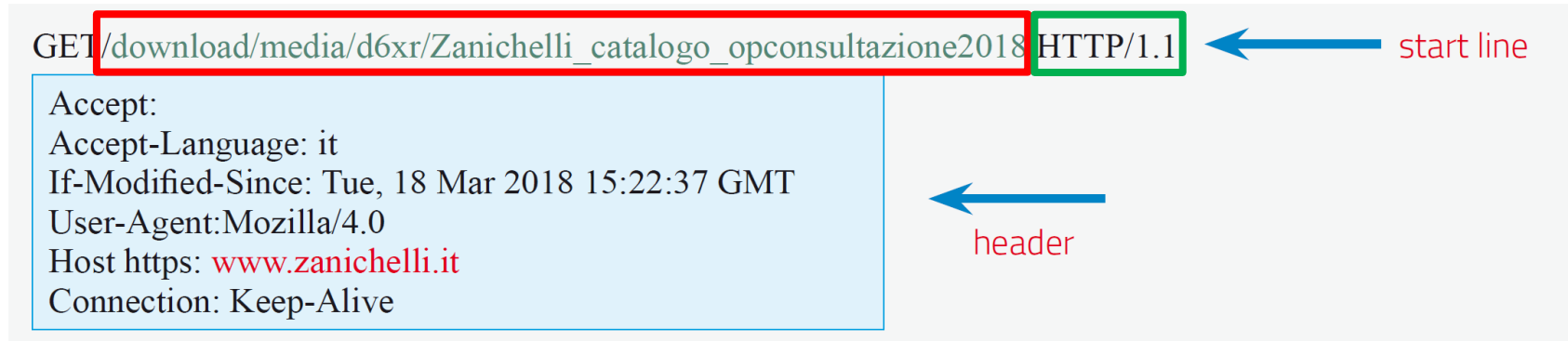
La **start line** contiene la richiesta del client ed è formata da tre parti: *metodo*, *URL* e *versione del protocollo*. Esistono diversi tipi di **metodo**, tra cui:

- **GET** per acquisire una risorsa presente sul server;
- **POST** per inviare dati al server;
- **HEAD** per acquisire dal server soltanto alcune informazioni su una risorsa (per esempio, la data dell'ultima modifica);
- **OPTIONS** per acquisire informazioni sul server, per esempio il tipo di server o i metodi di richiesta che supporta.

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).



Nella **start line** dopo il metodo si scrive l'**URL**, che specifica la risorsa del server richiesta dal client.

Infine la **versione del protocollo** è utile per concordare con il server quella da utilizzare nella comunicazione (HTTP 1.0 oppure 1.1).

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).

```
GET/download/media/d6xr/Zanichelli_catalogo_opconsultazione2018 HTTP/1.1
```

← start line

```
Accept:  
Accept-Language: it  
If-Modified-Since: Tue, 18 Mar 2018 15:22:37 GMT  
User-Agent: Mozilla/4.0  
Host https: www.zanichelli.it  
Connection: Keep-Alive
```

← header

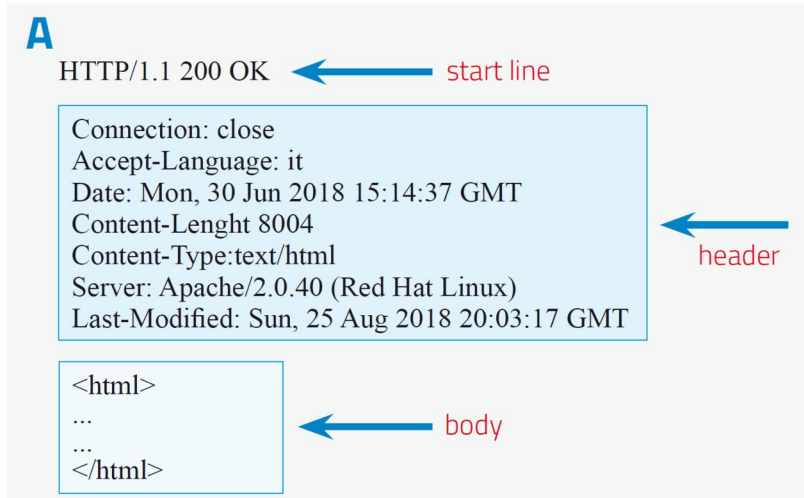
La sezione **header** dei messaggi di richiesta contiene diversi campi, tra cui:

- **If-Modified-Since**: la risorsa è richiesta soltanto se è stata modificata dopo una certa data;
- **User-Agent** fornisce al server informazioni sul browser usato dal client;
- **Host** è il campo in cui si inserisce il nome dell'host che si vuole raggiungere, per esempio **www.zanichelli.it**;
- **Connection** è un campo che può avere due valori: **close**, nel caso di connessione non persistente, o **keep-alive** nel caso contrario.

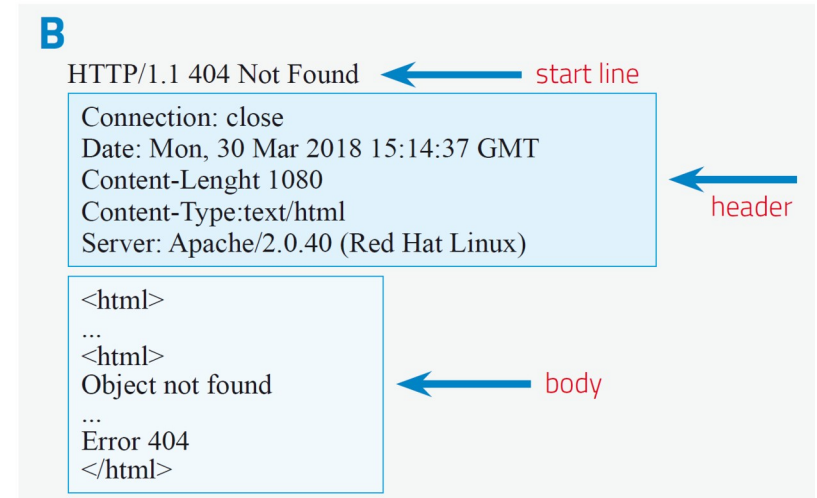
4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).



un esempio di risposta **positiva**
da parte del server



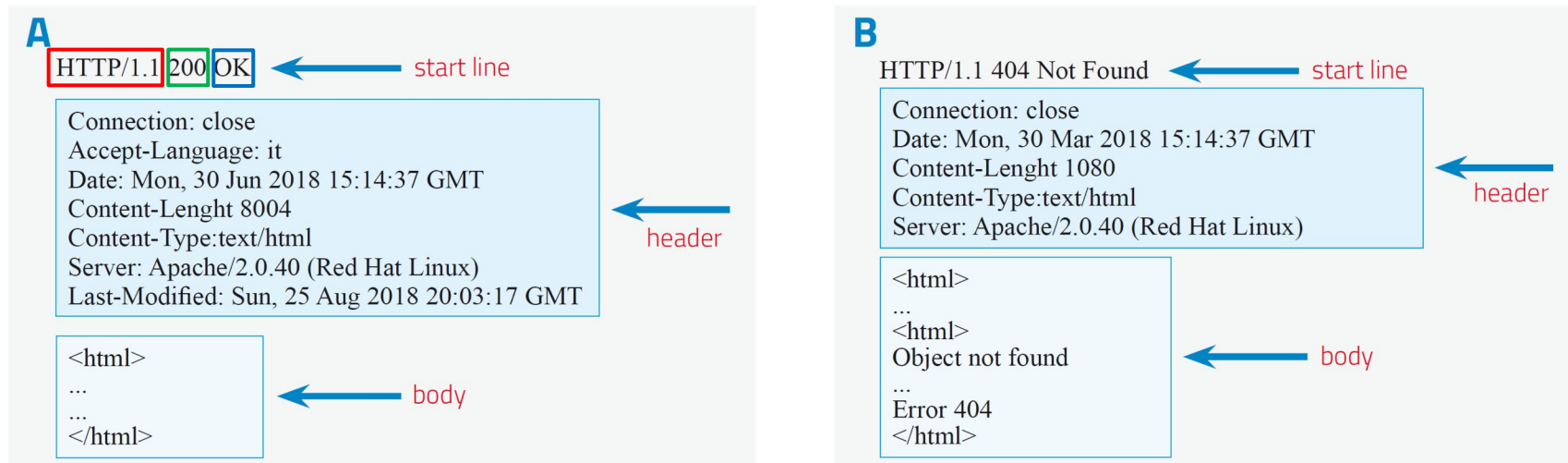
un esempio di risposta **negativa**
da parte del server

I messaggi di **risposta** dal server al client hanno invece **tre sezioni**: oltre a **start line** e **header** c'è anche la sezione **body**.

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).



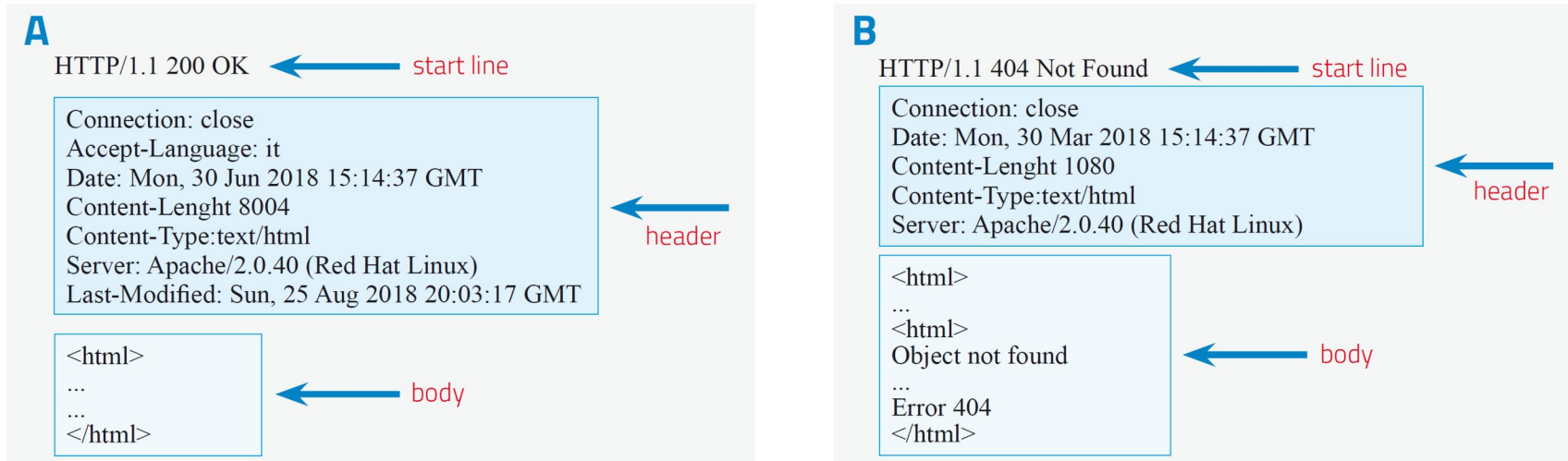
La sezione **start line** è formata da tre campi:

- la **versione del protocollo** ha la funzione già vista per le richieste
- il **codice di stato** dà informazioni sullo stato della richiesta; **esempi**: *richiesta ricevuta, capita e accettata* (codice **200**) o *risorsa non disponibile* (codice **404**)
- il **messaggio di stato**, una descrizione del codice di stato come **OK** o **Not Found**

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).



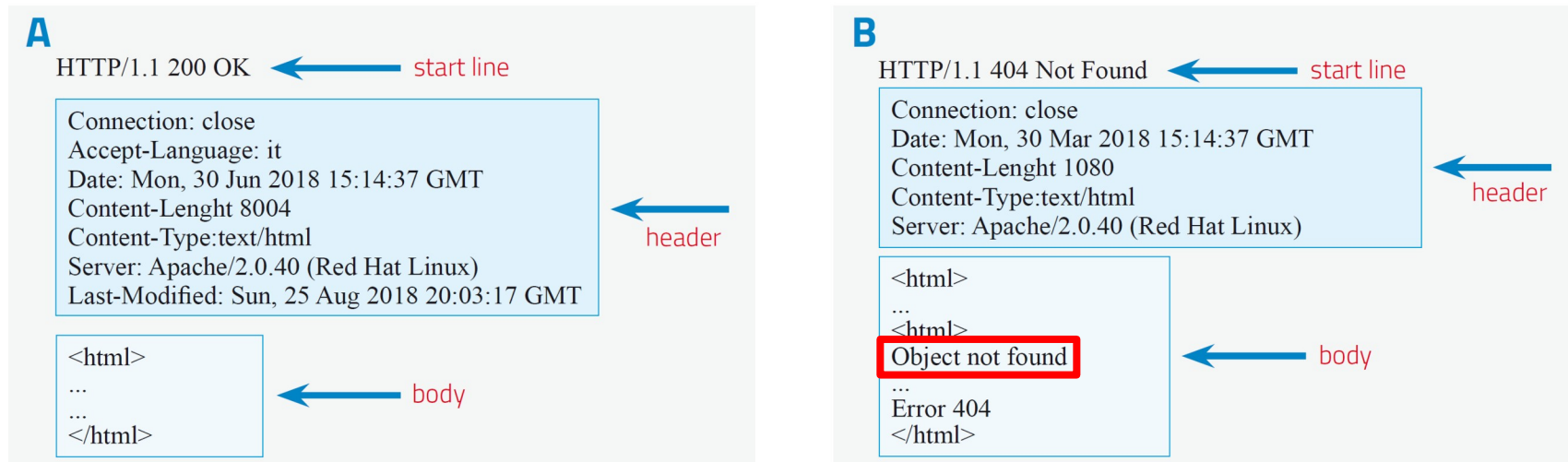
La sezione **header** dei messaggi di risposta contiene queste informazioni:

- **Connection** dà informazioni sulla connessione: **close** se sta per chiudersi;
- **Last-Modified** riporta data e ora dell'ultima modifica della risorsa richiesta;
- **Content-length** indica la lunghezza in byte dell'oggetto richiesto;
- **Content-Type** indica il tipo di oggetto: testi, immagini o altro.

4.3 Il protocollo HTTP

Indice

I **messaggi del protocollo HTTP** sono formati da **righe in caratteri ASCII** che terminano con il **carattere CR/LF** (*carriage return/line feed*).



La sezione **body**, cioè il corpo della risposta, contiene dati relativi all'oggetto richiesto dal client (per esempio il codice HTML, il codice CSS o un'immagine).

Nel caso in cui l'oggetto richiesto non esista nel server, il body del messaggio riporterà la dicitura **Object not found**.

4.3 Il protocollo HTTP

Indice

I **cookie**, «biscottini», sono **file di testo** contenenti **informazioni sulle precedenti navigazioni** di un particolare utente.

In molti casi è utile che il server tenga conto delle precedenti richieste del client.

Queste informazioni non sono memorizzate nel web server, bensì **nello stesso dispositivo dell'utente**.



Esempio:

1. l'utente si iscrive all'**area riservata di un sito**, inserendo il proprio nome-utente e la password, **memorizzati nel database del server**;
2. il server, nella risposta HTTP, inserisce un campo contenente l'**indicazione di creare un cookie** dove sia memorizzato un codice identificativo dell'utente;
3. il **browser** allora **crea e memorizza un cookie** che registra il codice indicato dal server e il nome del server;

4.3 Il protocollo HTTP

Indice

I **cookie**, «biscottini», sono **file di testo** contenenti **informazioni sulle precedenti navigazioni** di un particolare utente.

In molti casi è utile che il server tenga conto delle precedenti richieste del client.

Queste informazioni non sono memorizzate nel web server, bensì **nello stesso dispositivo dell'utente**.



4. quando l'utente si collegherà di nuovo all'area riservata del sito, il browser controllerà se sono presenti cookie relativi a quel server; in caso affermativo inserirà nel messaggio HTTP il **codice** contenuto nel cookie;
5. a quel punto il server, leggendo il codice, **identificherà l'utente** e potrà evitare di richiedergli le credenziali di accesso, che sono già memorizzate nel suo database.

4.3 Il protocollo HTTP

Indice

I **cookie sono utili**, perché **velocizzano le comunicazioni** tra client e server, ma presentano anche **rischi per la privacy**.

I grandi siti web commerciali e i gestori dei social network usano anche i cookie per **analizzare le abitudini, le preferenze e altri dati sensibili** relativi agli utenti, per distillare informazioni che poi possono rivendere ad altre aziende.

Con l'entrata in vigore nel 2018 del **Regolamento europeo GDPR sulla privacy**, è diventato obbligatorio per tutti i siti che usano cookie chiedere il consenso esplicito dell'utente.

I **browser** danno la possibilità di **bloccare completamente i cookie**; questa scelta però comporta la **rinuncia a tutte le loro funzioni utili**.

4.4 Trasferire file: il protocollo FTP

Indice

FTP è un'**applicazione di rete** che permette il **download e l'upload di file** da e verso un server web.

Il **servizio FTP** è usato soprattutto da chi gestisce siti web e ha bisogno di **inviare, modificare o cancellare file presenti sul server**, oppure dai portali web che offrono agli utenti l'opportunità di **scaricare file** con un servizio protetto da username e password.

Il **protocollo FTP** utilizza il **TCP a livello di trasporto**, in quanto ha bisogno di un **servizio affidabile**: un file che fosse trasferito soltanto parzialmente, o con errori, sarebbe infatti inutilizzabile dal dispositivo ricevente.

4.4 Trasferire file: il protocollo FTP

Indice

Per usare il servizio FTP, il client deve dotarsi di un'applicazione che fornisca un'interfaccia per l'utente, come il programma gratuito FileZilla.

Esempio: scaricata l'app dal sito ufficiale, dopo l'installazione nella barra

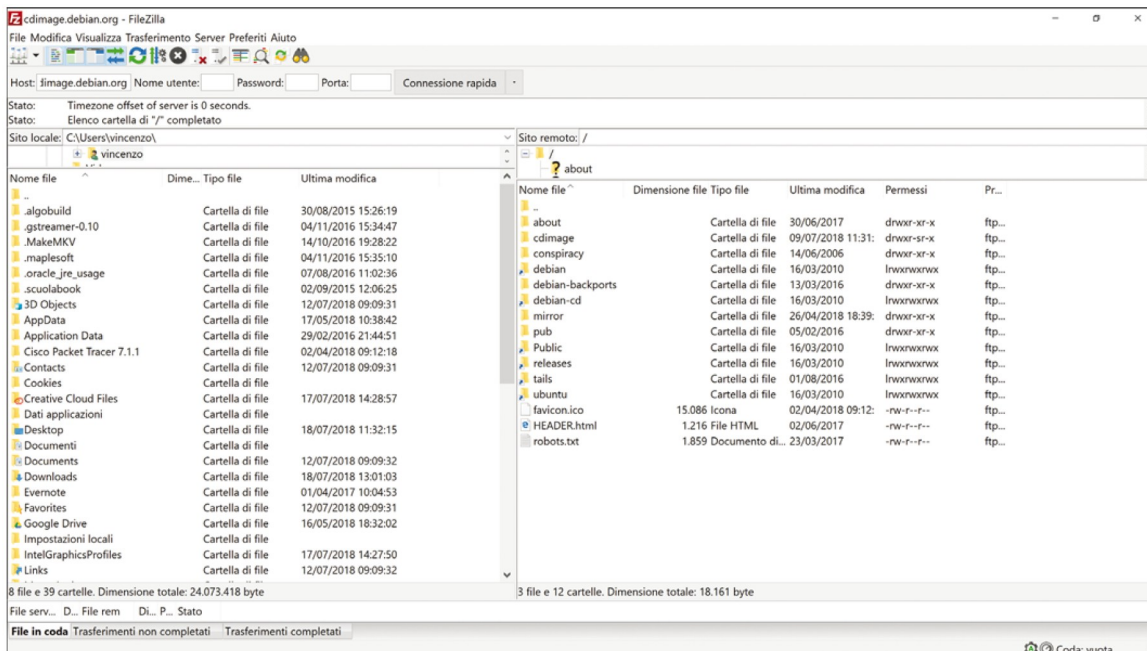
Connessione rapida in alto inserire le credenziali:

Host: *cdimage.debian.org*

Nome utente: *anonymous*

Password: *nulla*

Porta: *nulla*



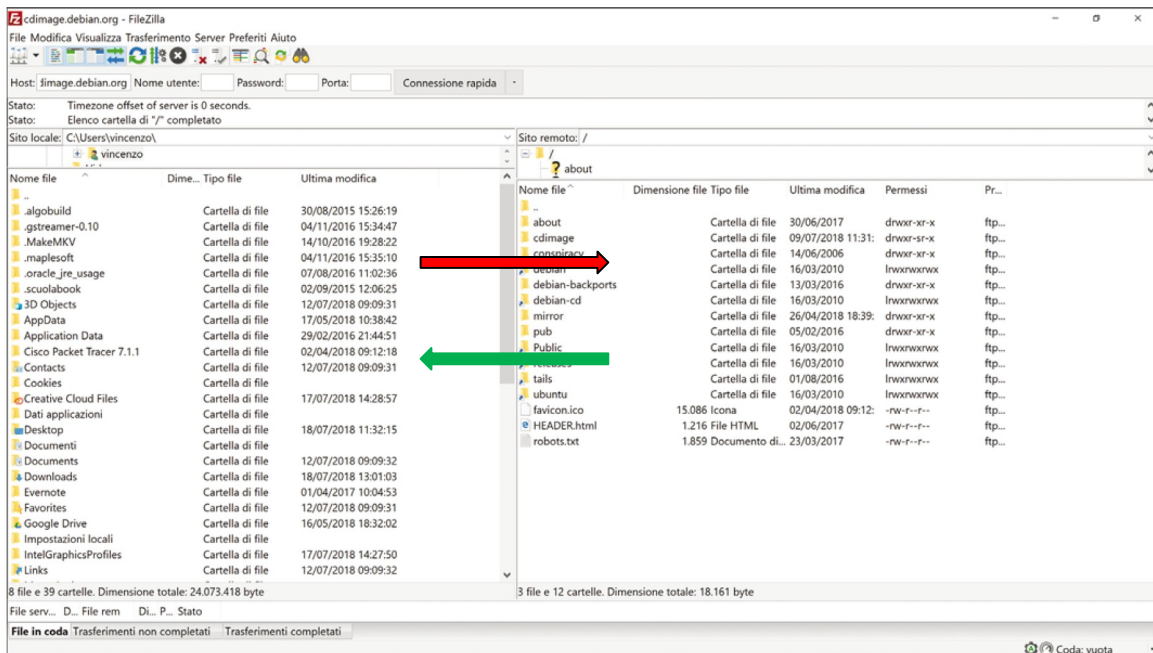
Dopo il clic sul pulsante **Connessione rapida**, nel riquadro di destra appariranno le cartelle e i file presenti sul sito **debian**, il portale dell'omonimo sistema operativo *open source*, che permette a qualsiasi utente di accedere ai file del server.

4.4 Trasferire file: il protocollo FTP

Indice

Per usare il servizio FTP, il client deve dotarsi di un'applicazione che fornisca un'interfaccia per l'utente, come il programma gratuito FileZilla.

In generale, invece, per accedere a un server remoto dotato di servizio FTP occorrerà specificare, oltre al nome dell'host e al nome-utente, anche la **password di accesso** e il **numero di porta**.



L'interfaccia permette di **creare nuove cartelle**, o di **riorganizzarle**, proprio come se fossero sul nostro computer. Per trasferire file e cartelle dal client al server, o viceversa, basta **trascinarli con il mouse** da un riquadro all'altro.

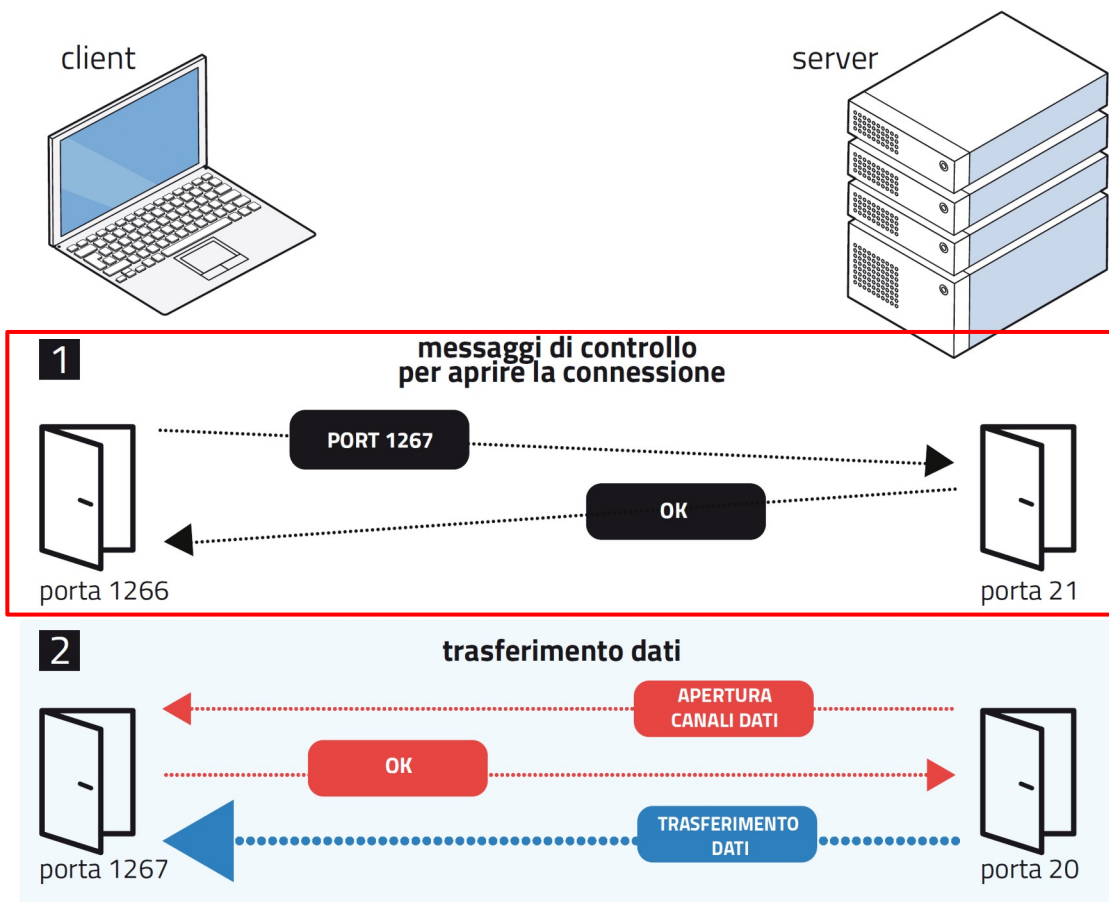
4.4 Trasferire file: il protocollo FTP

Indice

L'applicazione **FTP lato server** usa **due porte**: la **numero 21** per i **messaggi di controllo** della connessione e la **numero 20** per il **trasferimento dei dati**.

Un esempio di comunicazione tra un client e un server FTP nella **modalità normale**:

1. Il client attiva la connessione **con la porta numero 21** del server usando la propria **porta 1266** (casuale) e comunica al server che intende ricevere il file sulla **porta 1267** (casuale)

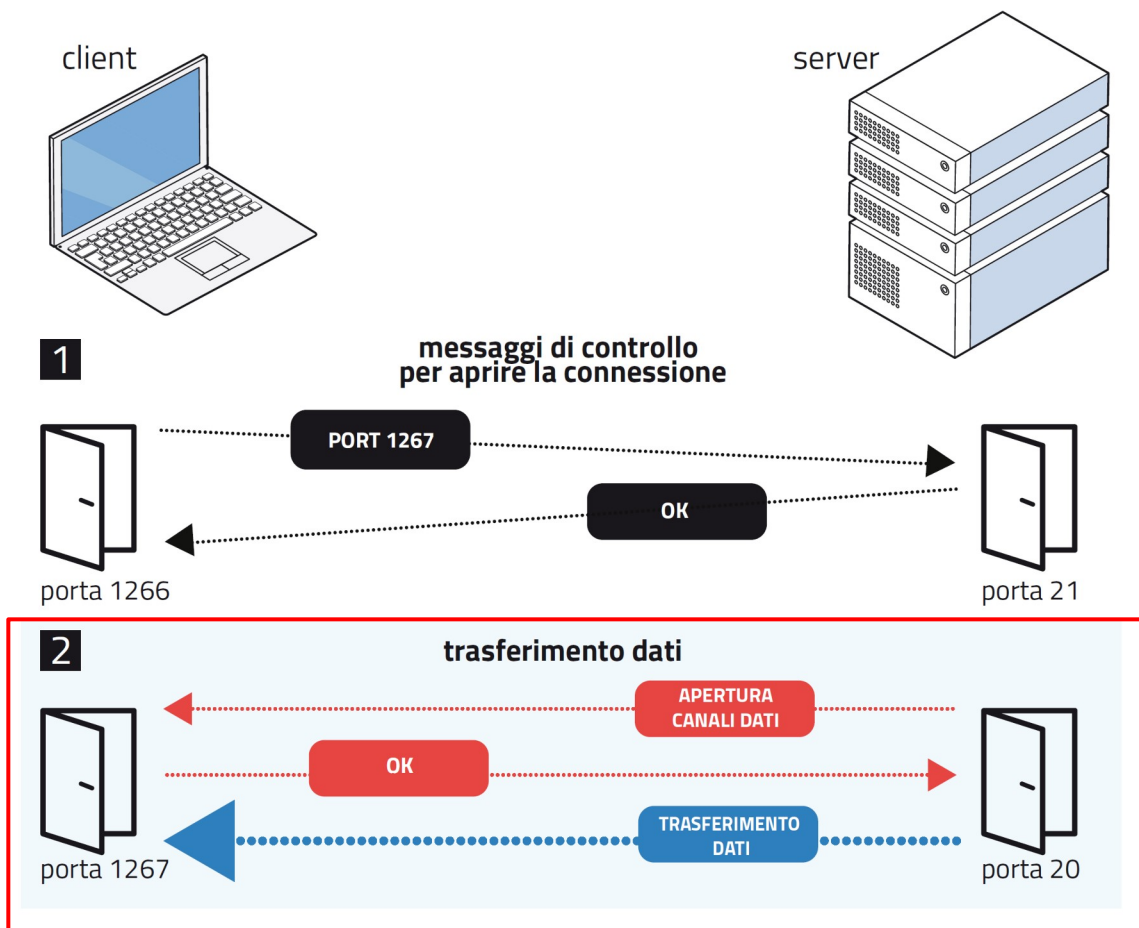


4.4 Trasferire file: il protocollo FTP

Indice

L'applicazione **FTP lato server** usa **due porte**: la **numero 21** per i **messaggi di controllo** della connessione e la **numero 20** per il **trasferimento dei dati**.

Un esempio di comunicazione tra un client e un server FTP nella **modalità normale**:



2. il server apre il canale dati sulla **porta 1267** ed **effettua il trasferimento** del file richiesto; a fine trasferimento, il **server chiude la connessione dati** sulla **porta 20**.

4.4 Trasferire file: il protocollo FTP

Indice

La **modalità normale è svantaggiosa** dal punto di vista del **client**.

Il **firewall del client** potrebbe **bloccare il tentativo del server** di instaurare una **connessione dati sulla porta 1267**, scelta in modo random dal client stesso, rendendo impossibile il trasferimento del file.

D'altra parte, **se il firewall non intervenisse, un malintenzionato potrebbe intercettare il messaggio** con cui il client comunica al server il proprio numero di porta dati (1267 nell'esempio) e penetrare nel suo sistema attraverso quella porta.

La modalità normale è invece **comoda per il server**: gli permette di **tenere aperte in ascolto le sole porte 20 e 21**, senza che vi possano essere problemi legati al firewall.

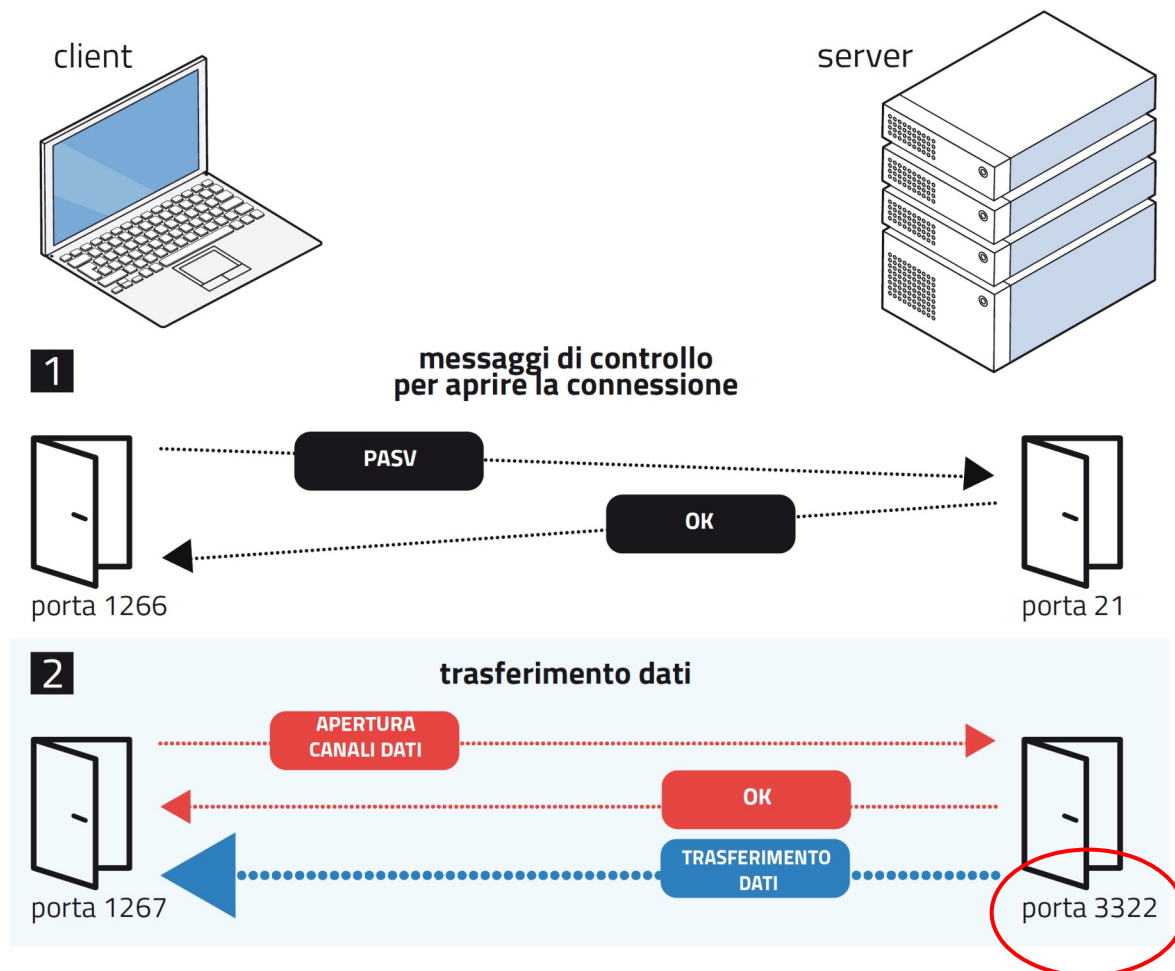
4.4 Trasferire file: il protocollo FTP

Indice

In alternativa alla modalità normale, il **client** può chiedere al server di **attivare la connessione in modalità passiva**.

Un esempio di comunicazione tra un client e un server FTP nella **modalità passiva**:

In questo schema è il **server a comunicare al client** il numero della propria porta su cui dovrà avvenire il trasferimento dati; questa porta (qui 3322) sarà **diversa dal numero 20 standard**.



4.4 Trasferire file: il protocollo FTP

Indice

La **modalità passiva**, che è di default su FileZilla, **è vantaggiosa per il client**.

Un eventuale malintenzionato **non potrà carpire il numero di porta** che il client usa per i dati, perché **non viene comunicato al server**.

Ora è il client ad aprire il canale dati sulla porta 3322 del server, quindi **il problema di un possibile blocco da parte del firewall ora si pone per il server**.

Per evitare possibili inconvenienti, il server sceglie la porta di comunicazione dati a caso **tra una gamma limitata di numeri di porte**, dopo aver ordinato al suo firewall di lasciarle aperte.

4.4 Trasferire file: il protocollo FTP

Indice

FTP in origine non era stato pensato con attenzione alla **sicurezza dei dati**: molte **versioni successive** hanno cercato di **migliorare questo aspetto**.

Il **protocollo FTPS** è un'estensione dell'FTP che permette al client, con il comando **AUTH TLS**, di chiedere che la comunicazione sia criptata con il protocollo TLS (*Transport Layer Security*).

Alcuni server non permettono le comunicazioni con i client che non abbiano richiesto il TLS.

Il **protocollo SFTP** (*SSH File Transfer Protocol*) generalmente usa il SSH (*Secure Shell*) invece del TLS.

A differenza del protocollo FTPS, l'SFTP prevede la cifratura sia dei comandi sia dei dati, impedendo così che le password e le informazioni sensibili siano trasmesse senza protezione.

Il suo difetto è che non è compatibile con il protocollo FTP.

4.5 La posta elettronica

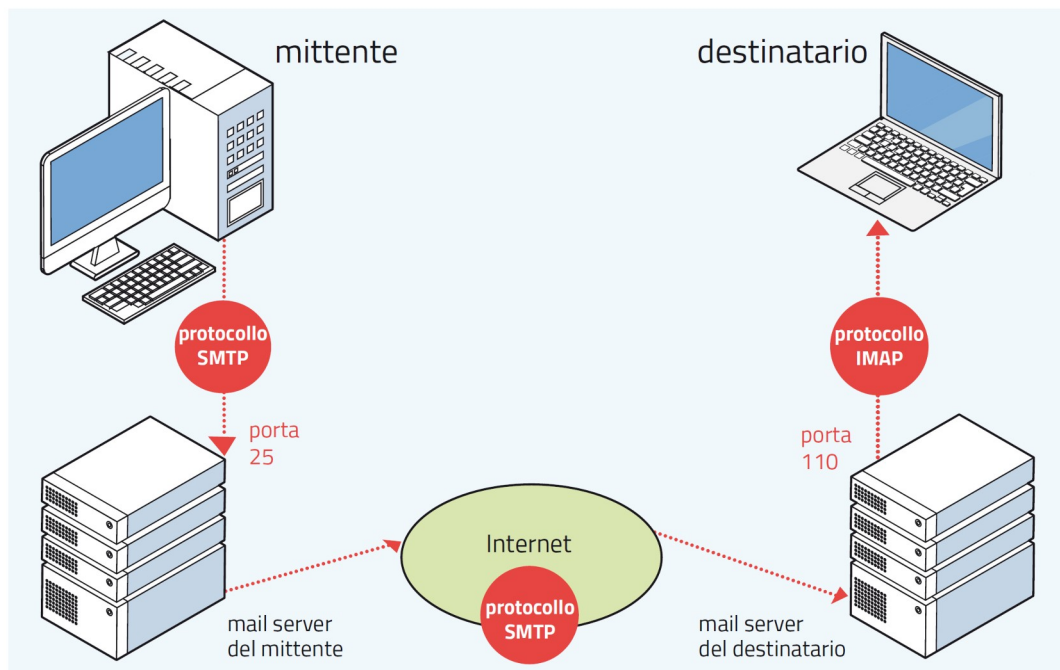
Indice

Un altro servizio di Internet è la **posta elettronica**, o **e-mail**, per lo scambio di **messaggi testuali e allegati di vario tipo**, come **documenti o immagini**.

Ogni utente può avere una o più **caselle di posta** o **mailbox**, ciascuna dotata di un indirizzo con la struttura *nome_utente@dominio*.

Esempio:
leonardodavinci@gmail.com

La **casella di posta** è uno spazio di memoria contenente un **file di tipo testo** in formato ASCII, che risiede in un **mail server** della rete e contiene tutte le e-mail di un utente.



Composto un messaggio, il mittente lo spedisce al proprio **mail server** specificando gli **indirizzi dei destinatari**, che individuano i mail server a cui consegnare il messaggio.

Il **client della posta** è l'applicazione con cui il mittente spedisce il messaggio dal proprio computer e il destinatario lo scarica sul proprio dispositivo.

Il ritiro della posta da parte del destinatario avviene tramite il protocollo **POP3** (*Post Office Protocol*) e la **porta 110**, oppure con il più recente protocollo **IMAP** (*Internet Message Access Protocol*) che opera attraverso la **porta 143**.

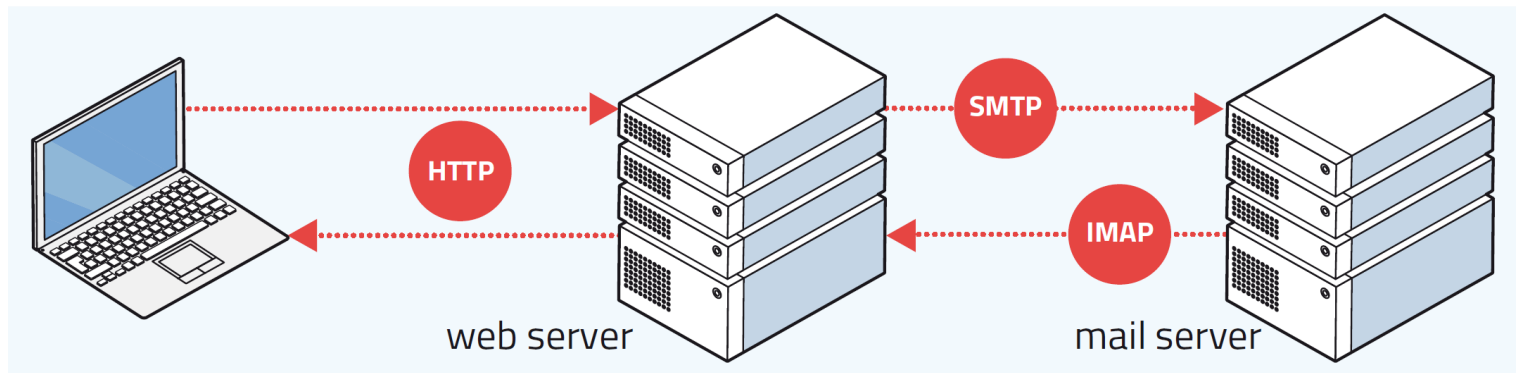
L'**IMAP** presenta numerosi vantaggi rispetto al POP3:

- i **messaggi rimangono sul server** e soltanto i più recenti sono memorizzati nella cache del client;
- si può accedere alla propria casella di posta da **molte postazioni diverse**, dalle quali i messaggi risulteranno sempre **sincronizzati**.

4.5 La posta elettronica

Indice

Oggi sono comuni i **sistemi *webmail*** – come Gmail, Hotmail o Yahoo – che **risiedono in Rete** e a cui si accede mediante il **browser**.



L'utente **compone e legge i messaggi direttamente sul sito del web server**, che con il protocollo HTTP fornisce un'interfaccia simile a quella di un client di posta tradizionale.

4.5 La posta elettronica

Indice

Il **protocollo SMTP** (*Simple Mail Transfer Protocol*) serve per **caricare i messaggi sul web server** e per la **comunicazione tra web server**.

Il protocollo SMTP è integrato dallo **standard MIME** (*Multipurpose Internet Mail Extensions*), che permette di associare ai messaggi di posta elettronica, come **allegati**, file di ogni tipo (documenti di testo, immagini, audio, video).

Gli **allegati** vengono codificati per **trasformarli in semplici file di testo**, in modo che possano essere gestiti come il messaggio principale. Il mail server del destinatario provvede poi a **decodificarli**, riportandoli al loro **stato originale**.

4.5 La posta elettronica

Il **protocollo SMTP** definisce la struttura del messaggio di posta elettronica, che è costituito da un **header** e da un **body**.

Nell'**header** sono contenute informazioni di controllo come:

To: destinatari

From: mittente

Cc: destinatari per conoscenza

Bcc: destinatari per conoscenza non visibili dagli altri destinatari

Date: data e ora di spedizione

Reply-to: indirizzo alternativo per rispondere al mittente

Subject: oggetto del messaggio

Un **documento MIME** prevede anche, tra gli altri, i campi seguenti:

MIME version: versione dello standard MIME usato nel messaggio

Content-Transfer-Encoding: tipo di codifica del contenuto

Content-Type: tipo di contenuto, per esempio audio o video

Content-ID: identificatore univoco del messaggio

Content-Description: descrizione del contenuto del messaggio

4.5 La posta elettronica

Indice

La comunicazione tra mittente e destinatario avviene tramite **comandi composti da una parola, da un numero di tre cifre (*reply code*) o da entrambi.**

I principali **comandi e reply code**:

comando	significato
HELO	serve per comunicare l'host name del client
MAIL FROM	dà inizio alla trasmissione indicando l'indirizzo del mittente
RCPT TO	identifica il destinatario (o i destinatari)
DATA	permette di spedire il corpo del messaggio (body)
SAML	spedisce l'e-mail
QUIT	termina la transazione
TURN	inverte i ruoli tra server e client
reply code	significato
221	chiusura della connessione
250	richiesta di azione completata, OK
354	inizia l'input dei dati
450	azione non avviabile (per esempio: mailbox piena)
500	errore di sintassi, comando non riconosciuto

Il **mail server** del mittente fa da **client**, inviando i **comandi** al mail server di destinazione.

4.5 La posta elettronica

Indice

Un **esempio di comunicazione tra mail server con le convenzioni del protocollo SMTP.**

```
220 smtp.gmail.com gsmtip server ready
HELO gmail.com
250 smtp.gmail.com hello [2.36.117.199], pleased to meet you
MAIL FROM: guglielmomaroni@gmail.com
250 ok
RCPT: galileogalilei@yahoo.it
250 ok
DATA
354
FROM: guglielmomaroni@gmail.com
TO: galileogalilei@yahoo.it
SUBJECT: incontro di domani
Ci vediamo alla solita ora
.

250 ok
QUIT
221 gmail.com
```

Dopo il messaggio **HELO** da parte del client, il server risponde con il **codice 250** accompagnato di solito da una **frase di benvenuto**, che in questo caso è **pleased to meet you.**

Il **browser** ha bisogno della **traduzione dei nomi simbolici in indirizzi IP**.

Per visualizzare una pagina web digitiamo nella **barra degli indirizzi del browser** un **nome simbolico**, che è facile da memorizzare e richiama il contenuto della pagina.

Il **browser** però ha bisogno di sapere l'**indirizzo IP del server** dove risiede il **sito**, indispensabile per stabilire una **connessione TCP/IP**.

Perciò per prima cosa il browser chiama un programma detto **risolutore** (*resolver*); questo interroga il servizio chiamato **DNS** (*Domain Name System*), che risiede su server predefiniti della Rete detti anche *default name server*.

Il DNS provvede a «**risolvere**» il **nome**, cioè a **tradurlo nell'indirizzo IP** corrispondente.

4.6 Il DNS

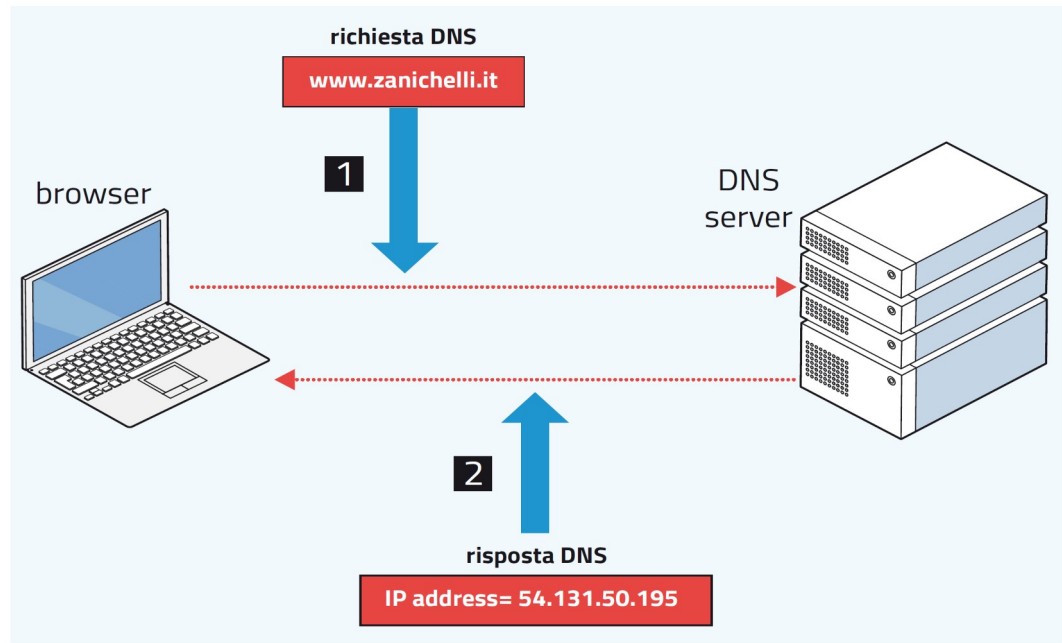
Indice

Il **browser** ha bisogno della **traduzione dei nomi simbolici in indirizzi IP**.

Esempio: l'utente digita l'indirizzo **www.zanichelli.it**

Il **DNS server** risponde con l'indirizzo **IP 54.131.50.195**.

A questo punto il browser può finalmente iniziare la comunicazione con il server che ospita il sito, utilizzando il protocollo HTTP.



4.6 Il DNS

Indice

I **nomi simbolici (URL)** dei siti web hanno una **struttura ad albero**.

Esempio: l'ipotetico URL di un sito dedicato alle collane di libri della Zanichelli:

www.collane.zanichelli.it

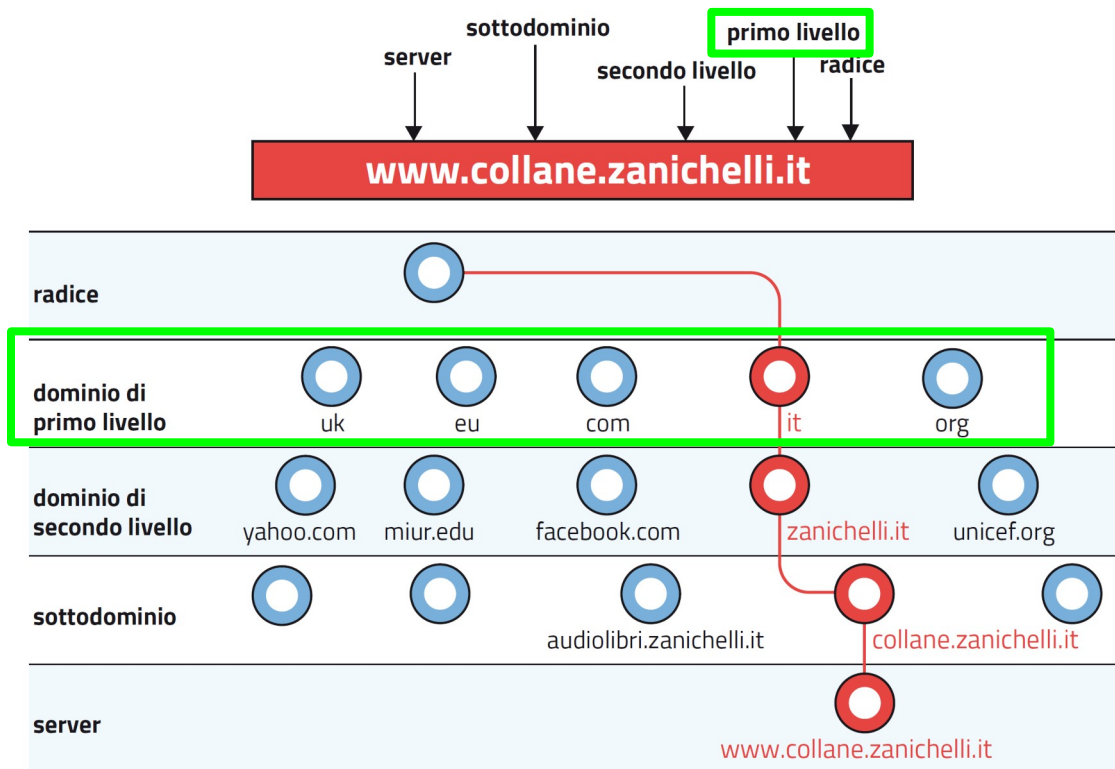
La gerarchia è tale che **il livello più alto sta all'estrema destra**; si scende di livello ogni volta che, spostandosi verso sinistra, si scavalca un punto « . ».

In realtà alla destra di **it** esiste un altro punto, normalmente omissso: a destra di **it** c'è il livello più alto, che è detto **radice** (*root*).

4.6 Il DNS

Indice

I **nomi simbolici (URL)** dei siti web hanno una **struttura ad albero**.



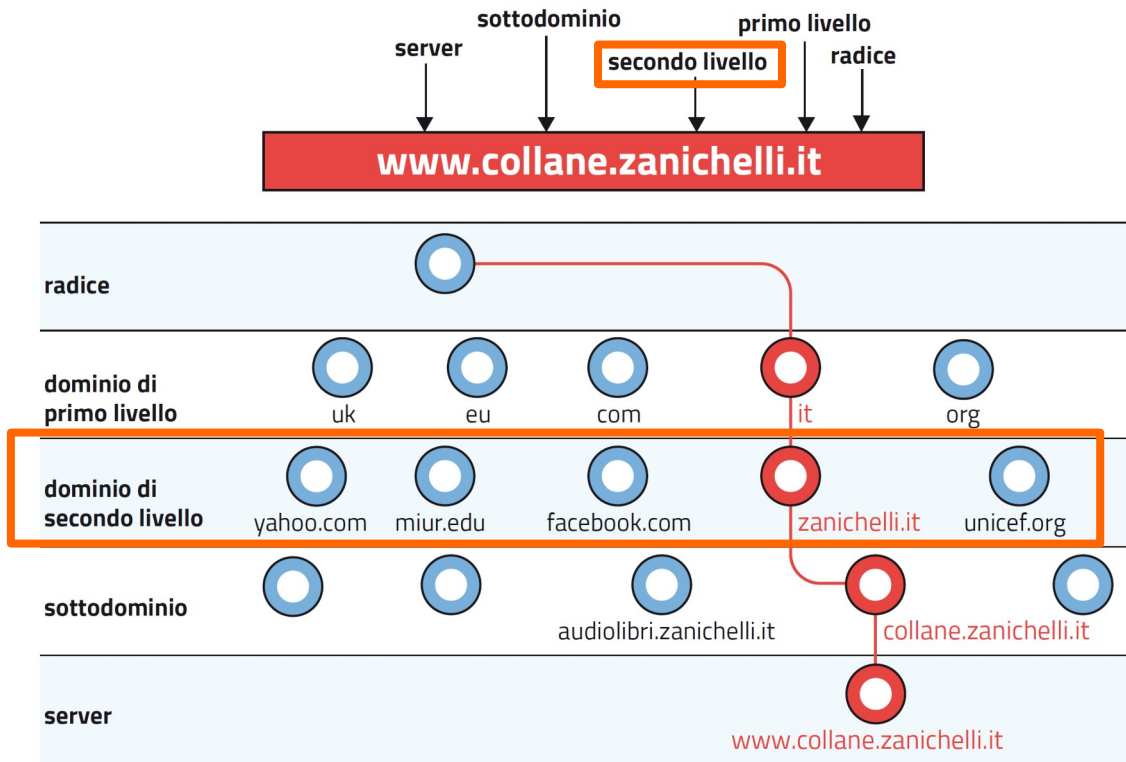
Procedendo verso sinistra dalla radice troviamo **it**, nome che fa parte del **dominio di primo livello** (*top level domain* o TLD).

Nel dominio di primo livello la **sigla** definisce un'organizzazione (come **com**, **org** o **gov**), un'area geografica (come **it** per l'Italia o **fr** per la Francia) oppure un'area generica (come **info**, **biz** o **museum**).

4.6 Il DNS

Indice

I **nomi simbolici (URL)** dei siti web hanno una **struttura ad albero**.

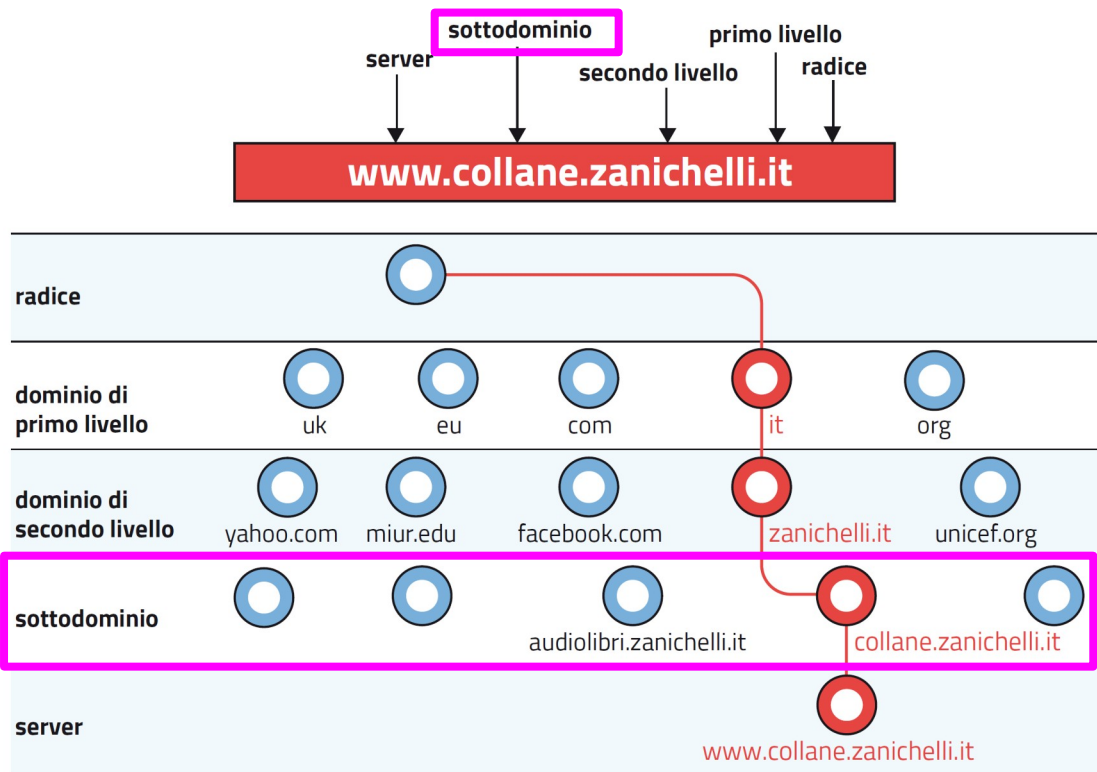


Ancora verso sinistra troviamo **zanichelli.it**, che fa parte del **dominio di secondo livello**, caratterizzato da nomi di aziende, associazioni o altre organizzazioni.

4.6 Il DNS

Indice

I **nomi simbolici (URL)** dei siti web hanno una **struttura ad albero**.

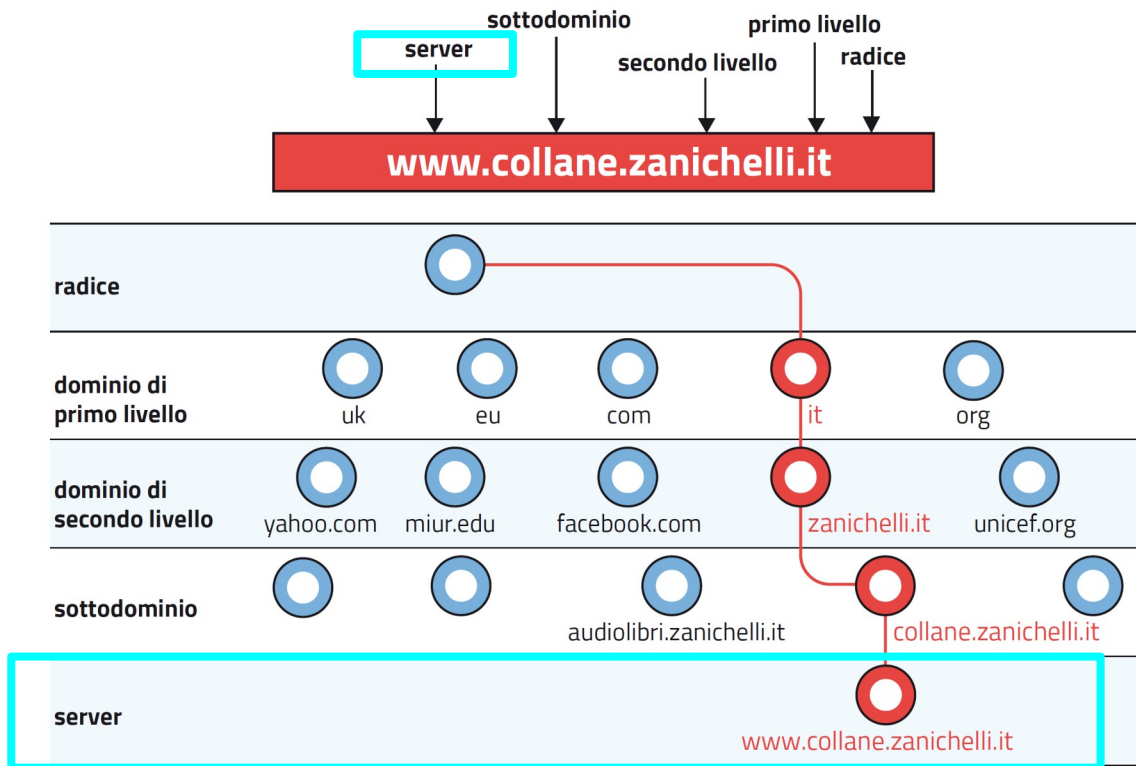


Un ulteriore passo verso sinistra ci porta a **collane.zanichelli.it**, nome che fa parte di un cosiddetto **sottodominio**; ogni dominio di secondo livello può avere molti diversi sottodomini.

4.6 Il DNS

Indice

I **nomi simbolici (URL)** dei siti web hanno una **struttura ad albero**.



Infine si arriva all'URL **www.collane.zanichelli.it**, che è il nome del server in cui risiede la pagina.

La rete di **server DNS** è organizzata sulla **gerarchia dei domini**.

Per ogni livello di **dominio** (radice, primo livello, secondo livello) ci sono server che contengono le informazioni relative al **livello sottostante**.

I server DNS del livello radice, chiamati **server radice**, sono **13 in tutto il mondo**. Sono server **detti autoritativi**, perché contengono le informazioni su tutti i server DNS dei domini di primo livello.

I server autoritativi di primo livello o **server TLD**, a loro volta, contengono le informazioni relative ai server di secondo livello, chiamati **server di competenza**.

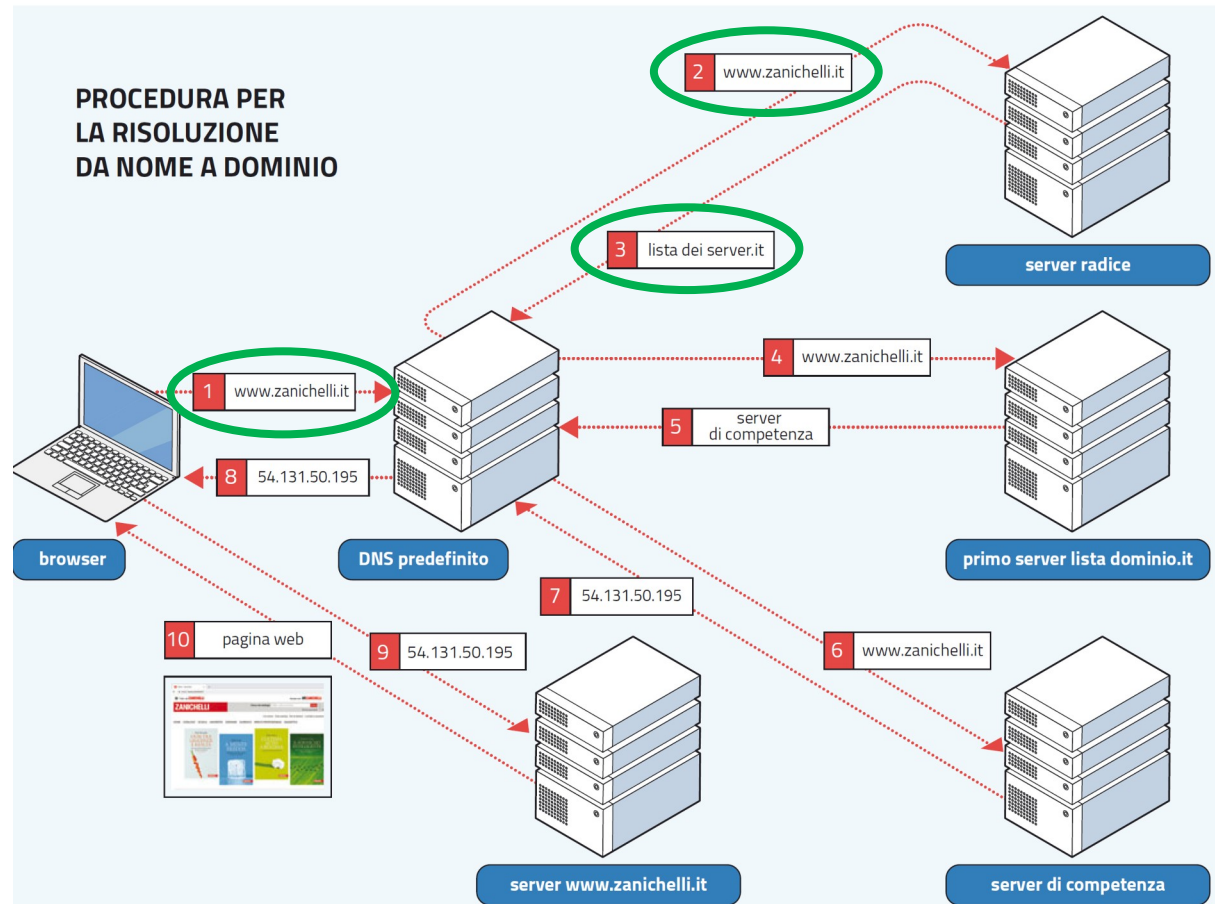
I server di competenza memorizzano i **record DNS** delle organizzazioni che hanno server accessibili pubblicamente in Internet.

Un record DNS contiene, insieme con altre informazioni, **la corrispondenza tra il nome simbolico e l'indirizzo IP dei server di un'organizzazione**.

4.6 Il DNS

La procedura iterativa per **risolvere da nome a dominio www.zanichelli.it**.

1. il browser invia la richiesta al server DNS predefinito
2. se questo server non conosce l'indirizzo IP del nome richiesto, effettuerà a sua volta un'interrogazione a un DNS server radice
3. il server radice risponde con l'elenco di tutti i server che si occupano del nome **.it**



4.6 Il DNS

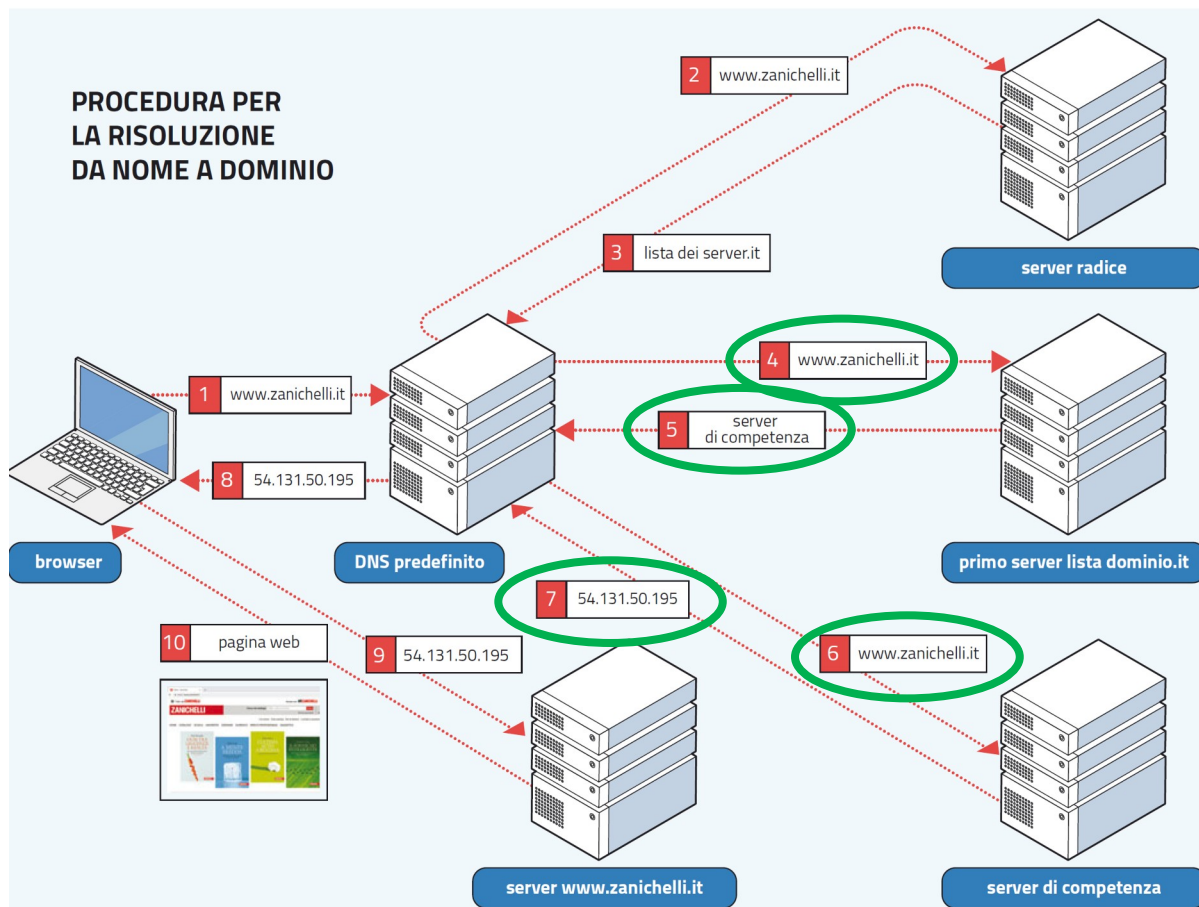
La procedura iterativa per **risolvere da nome a dominio www.zanichelli.it**.

4. il DNS predefinito interroga allora il primo server DNS della lista

5. questo server risponde indicando a quale server DNS di competenza ci si deve rivolgere

6. il DNS predefinito interroga il server DNS di competenza

7. quest'ultimo riesce a risolvere l'indirizzo IP e lo comunica al DNS predefinito



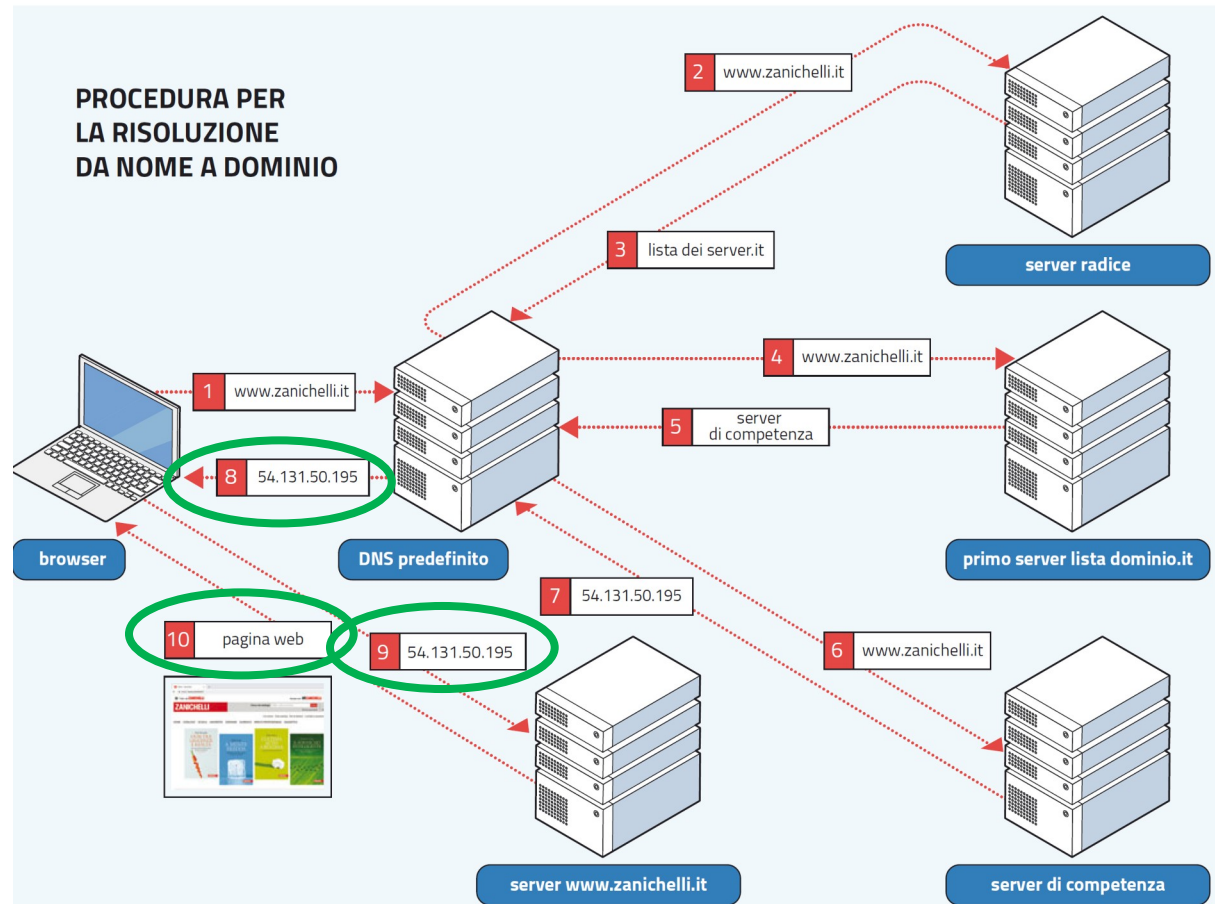
4.6 Il DNS

La procedura iterativa per **risolvere da nome a dominio www.zanichelli.it**.

8. il DNS predefinito comunica al browser del client l'indirizzo IP richiesto

9. ricevuta questa risposta, il browser può finalmente interrogare con il protocollo HTTP il web server cercato

10. il server Zanichelli ora potrà rispondere inviando al client i dati della pagina richiesta.



La struttura gerarchica dei server **DNS** serve per evitare la vulnerabilità del ***single point of failure***.

Qualsiasi dispositivo collegato alla Rete può richiedere pagine web di qualsiasi sito in **tutto il mondo**.

Se ci fosse un **unico server DNS**, esso dovrebbe **memorizzare tutti gli indirizzi IP del mondo** e dovrebbe sopportare un numero enorme di richieste simultanee.

Ciò richiederebbe una **memoria di capacità enorme**.

Inoltre un guasto a quell'unico server causerebbe il **collasso dell'intera Internet**. Questo tipo di vulnerabilità si chiama **SPOF**, ***single point of failure***.